

Discrete Mathematics 1 Lectures, Part 1

Tomasz Brengos

Committers: Aliaksei Kudzelka, Mykhailo Moroz, Mihail Orlov

*“There’s no such thing as a lousy job — only
lousy men who don’t care to do it.”*

AYN RAND

Preface

These lecture notes have been compiled from official course materials and lectures of Tomasz Brengos with contributions from students. They aim to provide a clear and comprehensive introduction to Discrete Mathematics 1. Although every effort has been made to ensure accuracy, some errors may remain. Your feedback and contributions are highly valued.

License

This work is licensed under the [Creative Commons Attribution-NonCommercial 4.0 International License](#). See the LICENSE file for details.

Acknowledgments

We gratefully acknowledge all the contributors whose efforts have enriched these notes.

Contents

1	Counting (Combinatorics)	4
1.1	Rule of Sum (Addition Principle)	4
1.2	Rule of Product (Multiplication Principle)	5
1.3	Rule of Bijection	5
1.4	Counting in Two Ways	5
1.5	Binomial Coefficients and Permutations	6
1.6	Binomial Theorem	7
1.7	Multisets	7
1.8	Lattice Paths	7
2	Partitions and Stirling Numbers	8
2.1	Set Partitions	8
2.2	Stirling Numbers of the Second Kind	8
2.3	Counting Maps	9
2.4	Number Partitions	10
3	Inclusion-Exclusion Principle	13
4	Permutations and Derangements	13
4.1	Permutations	13
4.2	Derangements	14
5	Preparation Tasks 1	15
5.1	Task 1	15
5.2	Task 2	17
5.3	Task 3	20
6	Preparation Tasks 2	24
6.1	Problem 1	24
6.2	Problem 2	25
7	Functions Between Sets	26
8	Generating functions	27
8.1	Generating Series	27
8.2	Building Generating Functions	28
8.3	Recurrence Relations & Generating Functions	29
8.4	Introduction to the Fibonacci Sequence	30
8.5	Deriving the Closed-Form for the Fibonacci Sequence	31
8.6	More examples	33
8.7	Generating Function Applications	36
9	Generating functions tutorial tasks	39
10	Catalan Numbers	47
10.1	Introduction and Motivating Problem	47
10.2	Definition of Catalan Numbers	47
10.3	Catalan Numbers and Binary Trees	48
10.4	Deriving the Formula using Generating Functions	49

Contents

11 Encoding and Decoding Functions	50
11.1 Encoding and Decoding Functions	50
11.2 Error Vectors and Transmission Errors	50
11.3 Examples of Encoding Functions (Codes)	51
11.4 Hamming Distance	52
11.5 Metric Spaces and Other Distance Metrics	53
11.6 Balls and Error Correction	53
11.7 A Geometric Example: $(\mathbb{Z}_2)^3$ as a Hamming Space	54
11.8 Error Detection and Correction Bounds	55
11.9 Detection, Correction, and Perfect Codes	56
11.10 Linear Codes	59
12 Introduction in Graph Theory	63
12.1 Formal definition	63
12.2 Basic Concepts and Terminology	64

1 Counting (Combinatorics)

Counting forms the basis of combinatorics. In these lectures we explore several counting rules, examples, and proofs.

1.1 Rule of Sum (Addition Principle)

If a set S is partitioned into disjoint subsets,

$$S = S_1 \cup S_2 \cup \cdots \cup S_k,$$

then the total number of elements in S is the sum of the number of elements in each subset:

$$|S| = |S_1| + |S_2| + \cdots + |S_k|.$$

Example: Suppose we wish to count the number of ways to choose a subset of a set X of size n , but we only consider subsets of a fixed size k . If we let S be the family of all such subsets, then using the rule of sum by dividing the choices according to a distinguished element (say, whether a chosen element is included or not) we can count the subsets by summing over the possibilities. (This idea is used later in proofs for binomial coefficients and the power set.)

Theorem:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

Proof: Consider $S = \binom{X}{k}$, the set of all subsets of X of size k . Take any element $a \in X$. Define: S_1 as the subsets in S that contain a and S_2 as the subsets in S that do not contain a .

Since every subset of S either contains a or does not, we see that S_1 and S_2 are disjoint and their union forms S , i.e.,

$$S_1 \cup S_2 = S.$$

By the rule of sum, we get:

$$|S| = |S_1| + |S_2|.$$

Now, each subset in S_1 must contain a , so we choose the remaining $k-1$ elements from $X \setminus \{a\}$, which has $n-1$ elements. Thus, $|S_1| = \binom{n-1}{k-1}$. Each subset in S_2 does not contain a , so we choose all k elements from $X \setminus \{a\}$. Thus, $|S_2| = \binom{n-1}{k}$.

Therefore,

$$\binom{n}{k} = |S| = |S_1| + |S_2| = \binom{n-1}{k-1} + \binom{n-1}{k}.$$

Example: Let $S = \{\triangle, \square, \circ\}$ and $k = 2$, choosing $a = \circ$. Fixing $\circ$ as one of the elements in the subset of size k , we get:

$$S_1 = \{\{\circ, \triangle\}, \{\circ, \square\}\}.$$

Taking all subsets of size k without $\circ$:

$$S_2 = \{\{\triangle, \square\}\}.$$

We have $|S_1| = 2$ and $|S_2| = 1$, so $|S_1| + |S_2| = 3$.

On the other hand,

$$\binom{3}{2} = \frac{3!}{2!(3-2)!} = 3.$$

Thus, $|S| = |S_1| + |S_2| = 3$, verifying the identity.

1.2 Rule of Product (Multiplication Principle)

When an object is constructed by a sequence of choices, where:

- The first choice can be made in a ways,
- The second in b ways,
- ...

the total number of objects is the product:

$$a \times b \times \cdots$$

Example: A word of length n over the binary alphabet $\{0, 1\}$ is formed by choosing one of 2 possibilities for each position. Hence, there are

$$2^n$$

possible words.

1.3 Rule of Bijection

If there exists a bijection (a one-to-one and onto mapping) between two sets S and T , then they have the same number of elements:

$$|S| = |T|.$$

Example: Consider the power set of a set X , denoted by $\mathcal{P}(X)$. There is a natural bijection between $\mathcal{P}(X)$ and the set of binary sequences of length $|X|$: for each subset $A \subseteq X$, assign the sequence $(a_1, a_2, \dots, a_n)$ where

$$a_i = \begin{cases} 1, & \text{if } x_i \in A, \\ 0, & \text{if } x_i \notin A. \end{cases}$$

This shows that

$$|\mathcal{P}(X)| = 2^{|X|}.$$

1.4 Counting in Two Ways

Rule of Counting in Two Ways When two formulae enumerate the same quantity, they must be equal.

Example:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

Proof: Consider a lattice grid of size $(n+1) \times (n+1)$, defined as:

$$X = \{(i, j) \mid i, j \in \{1, 2, \dots, n+1\}\}.$$

Clearly, $|X| = (n+1)^2$.

Now, partition X into three subsets: - X_1 , the points strictly below the secondary diagonal. - X_2 , the points strictly above the secondary diagonal. - X_3 , the points on the secondary diagonal itself.

Since these three sets form a partition, we have:

$$|X| = |X_1| + |X_2| + |X_3|.$$

Observing their sizes:

$$|X_1| = |X_2| = 1 + 2 + \cdots + n, \quad |X_3| = n + 1.$$

Thus,

$$(n+1)^2 = 2(1+2+\cdots+n) + (n+1).$$

Rearranging, we get:

$$1+2+\cdots+n = \frac{(n+1)^2 - (n+1)}{2} = \frac{n(n+1)}{2}.$$

Hence, we have proven the formula:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}.$$

1.5 Binomial Coefficients and Permutations

Let X be a set with $|X| = n$.

Subsets: The number of ways to choose a k -subset of X is given by the binomial coefficient

$$\binom{n}{k}.$$

Permutations: A k -permutation of a set X of size n is a k -word over the alphabet X whose entries are distinct.

Theorem: There are exactly

$$n(n-1)(n-2)\cdots(n-k+1)$$

k -permutations of an n -set.

Question: How are k -permutations of an n -set related to k -subsets of an n -set?

Answer: The difference between a k -permutation and a k -subset is that a permutation is ordered, while a subset is not. To express a k -permutation in terms of a k -subset, we need to account for all possible arrangements of the elements, which is $k!$. Thus,

$$k\text{-permutation} = \binom{n}{k} \cdot k!$$

Expressing $\binom{n}{k}$ as

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

we obtain:

$$k\text{-permutation} = \frac{n!}{(n-k)!}.$$

Proof by Counting in Two Ways: Count the number of k -permutations of an n -set in two ways:

(1) Directly, by applying the rule of product:

$$n \times (n-1) \times \cdots \times (n-k+1) = \frac{n!}{(n-k)!}.$$

(2) First choose a k -subset (in $\binom{n}{k}$ ways) and then arrange it (in $k!$ ways), giving

$$\binom{n}{k} \cdot k!.$$

Equate these two counts to obtain the relation.

1.6 Binomial Theorem

For any x, y in a field and nonnegative integer n , the binomial theorem states:

$$(x + y)^n = \sum_{k=0}^n \binom{n}{k} x^k y^{n-k}.$$

Explanation: This theorem is a direct consequence of counting the number of ways to choose k copies of x (and the remaining $n - k$ copies of y) when expanding the product.

1.7 Multisets

Definition: A multiset of a set X of size n is a function

$$m : X \rightarrow \mathbb{N}$$

that assigns a non-negative integer to each element of X , representing its multiplicity in the multiset.

Example: Let $X = \{a, b, c\}$, and consider the multiset $\{a, a, b\}$. Then, the function m is given by:

$$m(a) = 2, \quad m(b) = 1, \quad m(c) = 0.$$

Question: What is the number of k -multisets of a set of size n ?

Theorem: The number of all k -multisets of an n -set is

$$\binom{n + k - 1}{k}.$$

Proof: Let X be the set of all k -multisets of an n -set. Let Y be the set of all distributions of k identical objects into n buckets.

Claim 1: There is a bijection from X to Y . Thus, by the rule of bijection, we have

$$|X| = |Y|.$$

Claim 2: Let Z be the set of all binary sequences of length $n + k - 1$ with exactly $n - 1$ ones (or equivalently, k zeros). There is a bijection from Y to Z . Hence,

$$|Y| = |Z| \Rightarrow |X| = |Z|.$$

Since the number of such binary sequences is given by

$$\binom{n + k - 1}{k},$$

we conclude that

$$|X| = \binom{n + k - 1}{k}.$$

1.8 Lattice Paths

Consider an $m \times n$ grid with lattice points at the intersections.

Problem: How many paths are there from $(0, 0)$ to (m, n) if one may only move right or up?

Solution: Every path consists of exactly m right moves and n up moves. Thus, a path can be represented as a sequence of $m + n$ moves, where we choose n positions (out of $m + n$) for the up moves. Hence, the number of paths is:

$$\binom{m+n}{n}.$$

Bijection Explanation: There is a bijection between the set of such lattice paths and the set of binary sequences of length $m + n$ with exactly n ones (representing the up moves).

2 Partitions and Stirling Numbers

2.1 Set Partitions

Definition: A set $\{A_1, A_2, \dots, A_k\}$ of subsets of N forms a *partition* of the set N if:

$$A_i \neq \emptyset, \quad A_i \cap A_j = \emptyset \text{ for } i \neq j, \quad \text{and} \quad N = A_1 \cup A_2 \cup \dots \cup A_k.$$

If a partition P of the set N is of size k then we say that P partitions N into k blocks.

Example: For $N = \{1, 2, 3, 4\}$, one partition into 2 blocks could be

$$\{\{1, 3\}, \{2, 4\}\}.$$

All possible partitions of a set N are denoted by $\Pi(N)$.

2.2 Stirling Numbers of the Second Kind

Question: How many k -partitions of an n -set are there?

Answer: Let $S(n, k)$ (or $\{n \ k\}$) denote the answer to our question, called the *Stirling number of the second kind*. We define the base cases as follows:

$$S(0, 0) := 1, \quad S(0, k) := 0 \quad \text{for } k > 0.$$

Theorem: The total number of set partitions of N is given by

$$|\Pi(N)| = \sum_{k=0}^{|N|} S(|N|, k).$$

Remark: The quantity

$$B(|N|) := |\Pi(N)| = \sum_{k=0}^{|N|} S(|N|, k)$$

is called the *Bell number*.

Partitions and Stirling Numbers

Example: Let $N = [5] = \{1, 2, 3, 4, 5\}$. List all possible 2-partitions of N .

Firstly, consider cases where the first subset contains only one element:

$$1|2345, \quad 2|1345, \quad 3|1245, \quad 4|1235, \quad 5|1234.$$

Now, consider cases where the first subset contains two elements:

$$\begin{aligned} &12|345, \quad 13|245, \quad 14|235, \quad 15|234, \\ &23|145, \quad 24|135, \quad 25|134, \quad 34|125, \quad 35|124, \quad 45|123. \end{aligned}$$

Since we are only interested in the contents of the two subsets (not their arrangement or order), we do not list cases where the first subset has three or four elements, as these would be overcounting. For example, the partitions $12|345$ and $345|12$ are considered the same.

Thus, all possible partitions have been listed, and their total number is 15. Therefore,

$$S(5, 2) = 15.$$

Note: Stirling numbers consider objects that we distribute as distinct, the boxes (subsets) as identical, and the size of subsets as known. Due to this, in the example, we did not consider the cases $12|345$ and $345|12$ as distinct.

Additionally, Bell's number counts all possible partitions, meaning the number of subsets k is not fixed but varies from 0 to $|N|$. This concept may seem similar to multisets; however, Bell's number treats objects being distributed as distinct and the boxes(subsets) as identical, while multisets treat objects as identical and boxes as distinct.

Recurrence Relation: These numbers satisfy the recurrence:

$$S(n, k) = S(n-1, k-1) + k S(n-1, k).$$

Proof: Let $N = [n]$ and P be the set of all k -partitions of N . We observe that $|P| = S(n, k)$, where $S(n, k)$ denotes the Stirling number of the second kind.

Consider an element $x \in [n]$. Define the following subsets of P : X_1 consists of partitions in P where x forms a singleton block, i.e., one of the subsets A_i in the partition $\{A_1, A_2, \dots, A_k\}$ is $\{x\}$. $X_2 = P \setminus X_1$, meaning X_2 consists of partitions where x is not a singleton block but instead belongs to one of the k subsets.

Now, we compute their cardinalities: Since X_1 consists of partitions where x is a singleton, the remaining $n-1$ elements must be partitioned into $k-1$ subsets. Thus,

$$|X_1| = S(n-1, k-1).$$

In X_2 , the element x is assigned to one of the k subsets after partitioning the remaining $n-1$ elements into k subsets. Thus,

$$|X_2| = k \cdot S(n-1, k).$$

By the rule of sum,

$$S(n, k) = |X_1| + |X_2| = S(n-1, k-1) + k \cdot S(n-1, k).$$

This completes the proof.

2.3 Counting Maps

Setup: Consider two finite sets N and R of sizes n and r respectively. We want to answer three main questions about the functions from N to R :

Q1: How many functions from N to R are there?

Answer: Each of the n elements of N can be mapped to any of the r elements of R . Hence, there are r^n possible functions in total.

Q2: How many injective (one-to-one) functions from N to R ?

Answer: To build an injective function, choose a distinct image in R for each element of N . Thus, the number of injective functions is

$$r \times (r-1) \times (r-2) \times \cdots \times (r-n+1) = \frac{r!}{(r-n)!}.$$

Q3: How many surjective (onto) functions from N to R ?

Answer: A function $f : N \rightarrow R$ is surjective if every element of R has a nonempty preimage. Equivalently, the sets

$$f^{-1}(y_1), \quad f^{-1}(y_2), \quad \dots, \quad f^{-1}(y_r)$$

form a partition of N into r nonempty blocks. Since there are $S(n, r)$ ways to partition N into r nonempty subsets (where $S(n, r)$ is the Stirling number of the second kind), and each partition can be labeled in $r!$ ways (assigning each of the r blocks to a different $y_i \in R$), the total number of surjections is

$$\text{Sur}(n, r) = r! \cdot S(n, r)$$

Corollary: Let N and R be finite sets with $|N| = n$ and $|R| = r$. Then the total number of functions from N to R can be expressed as

$$|\text{Map}(N, R)| = r^n = \sum_{k=0}^r \binom{r}{k} k! S(n, k).$$

2.4 Number Partitions

Definition. A *number partition* of $n \in \mathbb{N}$ is an expression

$$n = \lambda_1 + \lambda_2 + \cdots + \lambda_k,$$

where

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_k \geq 1.$$

Example. List all possible different number partitions of $n = 5$ into two summands:

$$5 = 4 + 1 \quad \text{and} \quad 5 = 3 + 2.$$

Question. How many k -partitions of n are there?

Answer. Define

$$P(n, k) = \left\{ (\lambda_1, \dots, \lambda_k) \mid n = \lambda_1 + \lambda_2 + \cdots + \lambda_k, \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_k \geq 1 \right\},$$

and let

$$p(n, k) = |P(n, k)|.$$

Moreover, set

$$P(n) = \bigcup_{k=1}^n P(n, k) \quad \text{and} \quad p(n) = |P(n)|.$$

An immediate observation is that

$$p(n) = \sum_{k=0}^n p(n, k).$$

Example. List all possible different number partitions of $n = 5$:

$$P(5) = \left\{ \{5\}, \{4, 1\}, \{3, 2\}, \{3, 1, 1\}, \{2, 2, 1\}, \{2, 1, 1, 1\}, \{1, 1, 1, 1, 1\} \right\},$$

with, for instance,

$$P(5, 1) = \{\{5\}\}, \quad P(5, 2) = \{\{4, 1\}, \{3, 2\}\}, \quad P(5, 3) = \{\{3, 1, 1\}, \{2, 2, 1\}\},$$

$$P(5, 4) = \{\{2, 1, 1, 1\}\}, \quad P(5, 5) = \{\{1, 1, 1, 1, 1\}\}.$$

To derive recursive formulas for $p(n)$ and $p(n, k)$, we introduce the notation

$$p(n, \leq k) \stackrel{\text{def}}{=} |P(n, \leq k)|, \quad \text{where} \quad P(n, \leq k) = \bigcup_{i=1}^k P(n, i).$$

- **Observation 1:** $P(n, \leq n) = P(n)$ and hence $p(n, \leq n) = p(n)$.
- **Observation 2:**

$$p(n, \leq k) = \sum_{i=1}^k p(n, i) = p(n, 1) + p(n, 2) + \cdots + p(n, k).$$

Theorem. There is a bijection

$$\Phi: P(n, k) \longrightarrow P(n - k, \leq k),$$

defined as follows.

Represent a partition

$$\lambda = (\lambda_1, \lambda_2, \dots, \lambda_k) \in P(n, k)$$

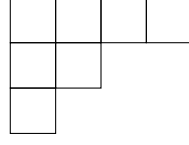
by its Ferrers diagram drawn in the standard way (with the largest row on top). In this diagram, the top row has λ_1 cells, the second row has λ_2 cells, and so on.

Explanation of the Mapping. Our goal is to relate partitions of n into exactly k parts to partitions of $n - k$ with at most k parts. Notice that if we subtract 1 from each part λ_i , then

$$n = \lambda_1 + \lambda_2 + \cdots + \lambda_k \implies n - k = (\lambda_1 - 1) + (\lambda_2 - 1) + \cdots + (\lambda_k - 1).$$

Graphically, subtracting 1 from a part corresponds to removing one cell from its corresponding row. Since the Ferrers diagram is left-justified, every row begins with a cell in the leftmost column. Thus, removing the entire leftmost column is equivalent to subtracting 1 from each λ_i . In this way, the original diagram representing a partition of n with k parts is transformed into a diagram representing a partition of $n - k$ that has at most k parts (some rows may vanish if $\lambda_i = 1$).

Example. Consider the partition $(\lambda_1, \lambda_2, \lambda_3) = (4, 2, 1)$ of $n = 7$ into 3 parts. Its Ferrers diagram is:



Ferrers Diagram for $(4, 2, 1)$

Removing the leftmost column yields:

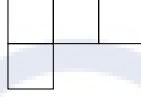


Diagram after removing leftmost column

The new diagram represents the partition $(3, 1)$, which is a partition of $7 - 3 = 4$ (since there were $k = 3$ rows, and we removed one cell per row). Note that $(3, 1)$ is an element of $P(4, \leq 3)$.

Justification of Bijectivity. The mapping

$$\Phi: (\lambda_1, \lambda_2, \dots, \lambda_k) \mapsto (\lambda_1 - 1, \lambda_2 - 1, \dots, \lambda_k - 1)$$

is invertible. Given any partition μ in $P(n - k, \leq k)$ (which has at most k parts), we can reconstruct a unique partition in $P(n, k)$ by adding 1 to each part and, if necessary, appending enough parts equal to 1 so that the total number of parts becomes exactly k . In other words, the inverse mapping Φ^{-1} is defined by:

$$\Phi^{-1}: \mu = (\mu_1, \mu_2, \dots, \mu_r) \mapsto (\mu_1 + 1, \mu_2 + 1, \dots, \mu_r + 1, \underbrace{1, 1, \dots, 1}_{k-r \text{ times}}),$$

with $r \leq k$. It is straightforward to check that Φ and Φ^{-1} are mutual inverses. Thus, the mapping Φ is a bijection, and we have the relation:

$$p(n, k) = |P(n, k)| = |P(n - k, \leq k)| = p(n - k, \leq k).$$

This bijective correspondence is the key step in obtaining a recursive formula for $p(n, k)$.

Corollary 1. *For all integers $n \geq k \geq 1$, we have*

$$p(n, k) = p(n - k, \leq k) = p(n - k, 1) + p(n - k, 2) + \dots + p(n - k, k - 1) + p(n - k, k).$$

That is, the total number of k -partitions of n can be split into partitions of $n - k$ with at most k parts:

$$p(n, k) = p(n - k, \leq k - 1) + p(n - k, k).$$

Moreover, since

$$p(n - k, \leq k - 1) = p((n - k) + (k - 1), k - 1) = p(n - 1, k - 1),$$

we obtain the recurrence relation

$$p(n, k) = p(n - 1, k - 1) + p(n - k, k).$$

3 Inclusion-Exclusion Principle

Very often, we need to calculate the number of elements in the union of certain sets. Assuming that we know the sizes of these sets, and their mutual intersections, the principle of inclusion and exclusion allows us to do exactly that.

Suppose you have two sets A and B . The size of the union is certainly at most $|A| + |B|$. However, in doing so we count each element of $A \cap B$ twice. To correct for this, we subtract $|A \cap B|$ to obtain

$$|A \cup B| = |A| + |B| - |A \cap B|.$$

In general, the formula gets more complicated because we must take into account intersections of multiple sets. The following statement is what we call the *principle of inclusion and exclusion*:

Lemma 1. *For any collection of finite sets $A_1, A_2, \dots, A_n$, we have*

$$\left| \bigcup_{i=1}^n A_i \right| = \sum_{\substack{I \subseteq [n] \\ I \neq \emptyset}} (-1)^{|I|+1} \left| \bigcap_{i \in I} A_i \right|.$$

Equivalently,

$$|A_1 \cup A_2 \cup \dots \cup A_n| = \sum_{i=1}^n |A_i| - \sum_{1 \leq i < j \leq n} |A_i \cap A_j| + \sum_{1 \leq i < j < k \leq n} |A_i \cap A_j \cap A_k| - \dots + (-1)^{n-1} |A_1 \cap A_2 \cap \dots \cap A_n|.$$

Proof Outline (informal): Each element that belongs to exactly t of the sets A_i is counted $\binom{t}{1}$ times in the first summation, then subtracted $\binom{t}{2}$ times in the second summation, added $\binom{t}{3}$ times in the third, and so on. In other words, its total contribution is

$$\binom{t}{1} - \binom{t}{2} + \binom{t}{3} - \dots + (-1)^{t-1} \binom{t}{t},$$

which equals 1. This alternating sum ensures that each element is ultimately counted exactly once, thereby correcting for any overcounting.

4 Permutations and Derangements

4.1 Permutations

A *permutation* of n elements is an arrangement (ordering) of those elements. For example, there are 6 permutations of the set $\{a, b, c\}$:

$$(a, b, c), \quad (a, c, b), \quad (b, a, c), \quad (b, c, a), \quad (c, a, b), \quad (c, b, a).$$

Since there are 3 choices for the first element, 2 for the second (once the first is chosen), and 1 for the last, by the multiplicative principle there are $3 \cdot 2 \cdot 1 = 3! = 6$ permutations in total.

Factorials and counting. In general, the number of permutations of n (distinct) elements is given by

$$n! = n \cdot (n-1) \cdot (n-2) \cdots 2 \cdot 1.$$

Partial permutations (k-permutations). Sometimes we only permute k of the n elements, where $1 \leq k \leq n$. The number of ways to do this is denoted $P(n, k)$ and can be found by thinking:

$$P(n, k) = n \times (n-1) \times \dots \times (n-k+1).$$

There are k factors in that product. Using factorial notation, we can write

$$P(n, k) = \frac{n!}{(n-k)!}.$$

Relationship to combinations. An alternate derivation uses combinations: first *choose* which k elements from the n will appear (that can be done in $\binom{n}{k}$ ways), then *arrange* those k in order (which can be done in $k!$ ways). Hence,

$$P(n, k) = \binom{n}{k} k!.$$

Since $\binom{n}{k} = \frac{n!}{(n-k)!k!}$, multiplying by $k!$ yields exactly $\frac{n!}{(n-k)!}$, consistent with the direct counting approach.

4.2 Derangements

A *derangement* of n elements is a permutation where no element remains in its original position. More precisely, if we think of a permutation as a bijection θ on the set $\{1, 2, \dots, n\}$, then θ is a derangement if and only if

$$\theta(k) \neq k \quad \text{for all } k \in \{1, 2, \dots, n\}.$$

Equivalently, a derangement has no fixed points.

For example, for $n = 3$, the permutations of $\{1, 2, 3\}$ are:

$$(1, 2, 3), \quad (1, 3, 2), \quad (2, 1, 3), \quad (2, 3, 1), \quad (3, 1, 2), \quad (3, 2, 1).$$

Among these, the derangements are $(2, 3, 1)$ and $(3, 1, 2)$; the other permutations fix at least one of the elements.

Counting Derangements via Inclusion-Exclusion

Let $D(n)$ denote the number of derangements of n elements. We will use the principle of inclusion-exclusion. Suppose we label the elements as $1, 2, \dots, n$, and define A_i to be the set of permutations that fix the element i (i.e. $\theta(i) = i$). Then any derangement is a permutation that lies in none of the sets A_i (for $1 \leq i \leq n$). We have

$$|A_i| = (n-1)!,$$

since if we fix one position i , then we permute the remaining $n-1$ elements freely. In general,

$$|A_{i_1} \cap A_{i_2} \cap \dots \cap A_{i_k}| = (n-k)!.$$

By inclusion-exclusion, the size of the union $A_1 \cup A_2 \cup \dots \cup A_n$ is

$$\sum_{k=1}^n (-1)^{k+1} \binom{n}{k} (n-k)!.$$

Hence the number of permutations that do not lie in this union—i.e. the number of derangements—is

$$D(n) = n! - \binom{n}{1}(n-1)! + \binom{n}{2}(n-2)! - \dots + (-1)^n \binom{n}{n}(n-n)!.$$

$$D(n) = \sum_{k=0}^n (-1)^k \binom{n}{k} (n-k)! = n! \sum_{k=0}^n \frac{(-1)^k}{k!}.$$

Thus, a concise closed-form for the number of derangements is

$$D(n) = n! \sum_{k=0}^n \frac{(-1)^k}{k!}.$$

Preparation Tasks 1

Note on the series for e^{-1} :

In Calculus, one learns that the exponential function has a power series expansion

$$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!}.$$

Setting $x = -1$ gives

$$e^{-1} = \sum_{k=0}^{\infty} \frac{(-1)^k}{k!}.$$

Hence,

$$\sum_{k=0}^n \frac{(-1)^k}{k!} \xrightarrow{n \rightarrow \infty} e^{-1}.$$

If you have not taken (or do not recall) a full course in Calculus, think of this as a special case of a well-known infinite series expansion for the exponential function.

Since the finite sum $\sum_{k=0}^n \frac{(-1)^k}{k!}$ converges to e^{-1} as $n \rightarrow \infty$, we conclude that

$$\lim_{n \rightarrow \infty} \frac{D(n)}{n!} = \lim_{n \rightarrow \infty} \sum_{k=0}^n \frac{(-1)^k}{k!} = e^{-1}.$$

Numerically, this means that for large n , about $1/e \approx 36.8\%$ of all permutations of $\{1, \dots, n\}$ are derangements (i.e. have no fixed points).

A Recurrence Relation

We can also show that $D(n)$ satisfies the recurrence

$$D(n) = (n-1)(D(n-1) + D(n-2)), \quad \text{with } D(1) = 0, D(2) = 1.$$

One way to see this: consider where 1 goes in a derangement of $\{1, 2, \dots, n\}$. It can go to any of $n-1$ positions. If 1 goes to position j , then either (i) the element j goes to position 1 (a swap), which reduces the problem to deranging the remaining $n-2$ elements, or (ii) the element j does *not* go to position 1, effectively reducing the problem to deranging $n-1$ elements. This yields the above recurrence.

5 Preparation Tasks 1

5.1 Task 1

Problem Statement:

There are 20 students and 5 trips. In each of the following scenarios, count the number of ways the students can be assigned to trips.

- (a) *Students are different, trips are different.* (I.e., we do care exactly which student goes to which distinct trip.)
- (b) *Trips are different, but we only care about how many students go to each trip.* (Not which specific students, only the counts per trip.)
- (c) *Trips are not distinguished, and we only care about the number of students in each trip.* (All trips are identical, so only the final “multiset of group sizes” matters.)

Preparation Tasks 1

- (d) *We do not care who goes where, but only with whom they go.* (We only care about the partition of the 20 students into some grouping, ignoring which trip is which.)
- (e) *Students are different, trips are different, and each trip has the same number of students.*

Solutions and Explanations

- (a) Each of the 20 **distinct** students independently chooses one of 5 **distinct** trips. This gives

$$5^{20}$$

total ways (each student has 5 choices).

- (b) Now the students' individual identities no longer matter; we only care that, for example, "Trip 1 has 7 students, Trip 2 has 3 students," etc. Since there are 5 **distinct** trips, we want the number of 5-tuples $(n_1, n_2, n_3, n_4, n_5)$ of nonnegative integers summing to 20:

$$n_1 + n_2 + n_3 + n_4 + n_5 = 20.$$

By the stars-and-bars formula, the count is

$$\binom{20 + 5 - 1}{5 - 1} = \binom{24}{4}.$$

- (c) Now the 5 trips themselves are **indistinguishable**, and we only care about how many students end up in each group (not which trip is which). Conceptually, this is the number of ways to partition 20 (distinct) students into up to 5 unlabeled subsets.

Interpreted as integer partitions, we want the number of partitions of the integer 20 into at most 5 parts. Denote by $p(n, i)$ the number of ways to partition n into exactly i (positive) parts. Then the total number of partitions of 20 into at most 5 parts is

$$p_{\leq 5}(20) = \sum_{i=1}^5 p(20, i).$$

- (d) We do not care who goes where, but only with whom they go. That is, we only care about how to group the 20 students, disregarding which trip label is attached to each group.

In terms of set partitions, the total number of ways to partition 20 distinct students into any number of unlabeled subsets is the Bell number B_{20} . The Bell number can be expressed as a sum of Stirling numbers of the second kind:

$$B_{20} = \sum_{k=0}^{20} S(20, k),$$

where $S(n, k)$ (with $S(n, 0) = 0$ for $n > 0$) counts the number of ways to partition n distinct elements into exactly k nonempty unlabeled subsets. Equivalently, one can start the sum at $k = 1$ since there are no partitions of 20 elements into 0 subsets:

$$B_{20} = \sum_{k=1}^{20} S(20, k).$$

Preparation Tasks 1

(e) Students are different, trips are different, and each trip has the *same* number of students. Since 20 is divisible by 5, that means exactly 4 students must go on each trip. Count the ways to split 20 distinct students into 5 distinct groups of 4 each. One way to see this is:

$$\underbrace{\binom{20}{4}}_{\text{Trip 1}} \underbrace{\binom{16}{4}}_{\text{Trip 2}} \cdots \underbrace{\binom{4}{4}}_{\text{Trip 5}}.$$

Equivalently, the multinomial coefficient

$$\frac{20!}{4! 4! 4! 4! 4!}.$$

5.2 Task 2

Problem Statement:

We have 6 toy cars and 4 dolls (total 10 toys), and 10 boxes. In each scenario, count how many ways there are to place the 10 toys into the 10 boxes, under various assumptions of distinctness or identicalness:

- (a) Everything (cars, dolls, boxes) is different.
- (b) Everything is different and every toy goes into a separate box.
- (c) All toys are different, but boxes are identical.
- (d) Cars are different, but all dolls are identical; boxes are different.
- (e) All cars are identical, all dolls are identical; boxes are different.
- (f) Cars are different, but all dolls are identical; boxes are identical.
- (g) We don't care which toy is which, but only how many toys are in each box; boxes are different.
- (h) We don't care which toy is which, but only how many toys are in each box; boxes are identical.

Solutions and Explanations

Let us denote our toys as follows:

$$\text{Cars : } C_1, C_2, C_3, C_4, C_5, C_6; \quad \text{Dolls : } D_1, D_2, D_3, D_4.$$

We have 10 boxes, say $B_1, B_2, \dots, B_{10}$.

(a) Everything is different. Each of the 10 distinct toys can go into any of 10 distinct boxes. Hence there are

$$10^{10}$$

ways (each toy has 10 choices independently).

(b) Everything is different and every toy goes into a separate box. We must place exactly one of the 10 distinct toys in each of the 10 distinct boxes (no box is left empty, no box has more than one toy). This is simply a permutation of 10 objects into 10 boxes:

$$10!$$

ways.

Preparation Tasks 1

(c) **All toys are different, but boxes are identical.** We have 10 *distinct* toys (labeled objects) to be placed into up to 10 *indistinguishable* boxes. Equivalently, this is the number of ways to partition a set of 10 labeled elements into any number of unlabeled subsets (possibly fewer than 10 if some boxes are empty).

The total count is the Bell number B_{10} . It can be expressed as a sum of Stirling numbers of the second kind:

$$B_{10} = \sum_{k=0}^{10} S(10, k) = \sum_{k=1}^{10} S(10, k),$$

where $S(n, k)$ is the number of ways to partition n distinct objects into k nonempty subsets. There is no simple closed form for B_{10} , so we typically leave the answer in this summation form or as B_{10} itself.

(d) **Cars are different, but dolls are identical; boxes are different.** - We have 6 distinct cars $C_1, \dots, C_6$. Each can go into any of 10 distinct boxes: 10^6 ways. - We have 4 *identical* dolls. Distributing 4 indistinguishable objects into 10 distinct boxes is given by the “stars-and-bars” formula:

$$\binom{4 + 10 - 1}{4} = \binom{13}{4}.$$

Multiply these independent choices:

$$10^6 \times \binom{13}{4}.$$

(e) **All cars are identical, all dolls are identical; boxes are different.** - Distribute 6 identical cars into 10 distinct boxes: $\binom{6 + 10 - 1}{6} = \binom{15}{6}$. - Distribute 4 identical dolls into 10 distinct boxes: $\binom{4 + 10 - 1}{4} = \binom{13}{4}$. Multiply for the total:

$$\binom{15}{6} \times \binom{13}{4}.$$

(f) We have a total of 10 toys:

$$\underbrace{C_1, C_2, C_3, C_4, C_5, C_6}_{6 \text{ distinct cars}} \quad \text{and} \quad \underbrace{D, D, D, D}_{4 \text{ identical dolls}},$$

which are to be placed into 10 identical boxes (i.e. unlabeled subsets). We want the number of different ways to partition these toys according to which cars appear together and how many dolls join them.

Step 1: Partition the Cars First, partition the 6 *distinct* cars into k nonempty subsets (blocks). The number of ways to do this is given by the Stirling number of the second kind

$$S(6, k).$$

Since each of the 6 cars is distinct, we have $S(6, 1)$ ways to partition them into 1 block, $S(6, 2)$ ways into 2 blocks, $\dots$, up to $S(6, 6)$ ways to put each car in its own block. Hence k ranges over $1 \leq k \leq 6$.

Step 2: Distribute the Dolls Next, we need to distribute the 4 *identical* dolls among these k car-blocks and possibly place some (or all) dolls in new subsets that contain only dolls.

Let

a = the number of dolls placed into the k car-containing blocks,

so that the remaining $4 - a$ dolls may form additional subsets, each of which contains *only* dolls (with no cars).

Preparation Tasks 1

Dolls Within the Existing k Blocks. Since the k blocks of cars are *distinct* (once the cars have been chosen to form those blocks), distributing a identical dolls among the k blocks is counted by the usual “stars and bars” formula:

$$\binom{a+k-1}{k-1}.$$

New Subsets of Only Dolls. Now, out of the $(4-a)$ remaining dolls, we can create any number t of subsets ($0 \leq t \leq 4-a$), each consisting of a positive number of dolls. Because these new subsets (boxes) are *unlabeled*, the number of ways to partition $(4-a)$ identical dolls into exactly t nonempty unlabeled subsets is

$$p(4-a, t),$$

where $p(n, k)$ denotes the number of ways to partition n identical items into k nonempty unlabeled subsets. Summing over all t from 0 to $(4-a)$ then gives the total ways to form any number of doll-only subsets out of the $(4-a)$ dolls.

Putting It All Together For a fixed value of k (the number of car-blocks), the number of ways to handle the dolls is

$$\sum_{a=0}^4 \left[\binom{a+k-1}{k-1} \times \sum_{t=0}^{4-a} p(4-a, t) \right].$$

We then multiply by $S(6, k)$, the ways to choose the k car-blocks in the first place. Finally, we sum over all possible k :

$$\text{Total ways} = \sum_{k=1}^6 \left[S(6, k) \times \sum_{a=0}^4 \left(\binom{a+k-1}{k-1} \times \sum_{t=0}^{4-a} p(4-a, t) \right) \right].$$

“However, this task should not appear on the test.”

(g) We don’t care which toy is which, only about how many toys are in each box; boxes are different. Now all 10 toys are treated as identical. With 10 *distinct* boxes, we only need the distribution of 10 identical items into 10 distinct bins, i.e. the number of solutions to

$$n_1 + n_2 + \cdots + n_{10} = 10$$

where $n_i \geq 0$. By stars-and-bars, that is

$$\binom{10+10-1}{10} = \binom{19}{10}.$$

(h) We don’t care which toy is which, only about how many toys are in each box; boxes are identical. Now both the toys and the boxes are considered identical. We want the number of ways to partition 10 identical objects among up to 10 identical boxes. Equivalently, this is the number of ways to express the integer 10 as a sum of positive integers, ignoring order.

We denote by $p(n)$ the total number of partitions of n . This can also be expressed in terms of $p(n, k)$, which is the number of ways to partition n into exactly k positive parts:

$$p(n) = \sum_{k=1}^n p(n, k).$$

Since we can’t have more than 10 parts if each part is positive, for $n = 10$ we write

$$p(10) = \sum_{k=1}^{10} p(10, k).$$

Preparation Tasks 1

It is known (by direct enumeration or from tables) that

$$p(10) = 42.$$

Hence, there are 42 ways to distribute 10 indistinguishable toys among 10 indistinguishable boxes.

Remark. Parts like (f) are more intricate when combining “partially identical” toys with “identical boxes.” Typically, such problems require detailed case analysis or generating functions.

5.3 Task 3

There are n rows with n chairs in each row (so n^2 chairs in total), and we have k students (where $k \leq n$) who are all distinct unless otherwise noted. We want to count the number of ways they can sit subject to various conditions. In each part, “at most one person in each chair” is always assumed.

(a) No extra assumptions.

We are simply placing k *distinct* students into n^2 distinct seats. First choose which k seats will be occupied, then permute the k students among those chosen seats. The number of ways is:

$$\binom{n^2}{k} k! = \frac{n^2!}{(n^2 - k)!}.$$

(b) Everyone sits in the first row.

The first row has n chairs and $k \leq n$ students. We must pick which k of the n seats are used, and then assign the k distinct students to those seats. Hence

$$\binom{n}{k} k!.$$

(c) We only care about the *numbers* of students sitting in each row.

Now we ignore which particular seats in each row are used *and* which particular students go to a given row. We only record the integer vector $(x_1, x_2, \dots, x_n)$ of row-occupancies, where x_i is the number of students in row i , and

$$x_1 + x_2 + \dots + x_n = k, \quad 0 \leq x_i \leq n.$$

Since $k \leq n$, the constraint $x_i \leq n$ is automatically satisfied. The number of nonnegative solutions to $x_1 + \dots + x_n = k$ is

$$\binom{k + n - 1}{n - 1}.$$

(Here the students themselves are no longer distinguished, nor are seats in the same row.)

(d) We only care about *which chairs* are taken, not who sits in them.

In this scenario, we ignore the identity of students and only note *the set of occupied seats* (of size k). Hence we need to choose exactly k chairs out of n^2 , with no regard to which student goes where. The count is

$$\binom{n^2}{k}.$$

(e) We only care about the numbers of students in each row, with each row having at most as many students as the previous row.

Let x_i be the number of students in row i . Then we record

$$(x_1, x_2, \dots, x_n) \quad \text{such that} \quad x_1 \geq x_2 \geq \dots \geq x_n \geq 0 \quad \text{and} \quad x_1 + x_2 + \dots + x_n = k.$$

Preparation Tasks 1

Equivalently, we are looking for a partition of k into at most n parts. Let

$$p(k, j)$$

be the number of partitions of k into exactly j positive parts. Then the number of partitions of k into at most n parts is

$$p_{\leq n}(k) = \sum_{j=0}^n p(k, j).$$

(Of course, $p(k, 0) = 0$ unless $k = 0$, but we often include $j = 0$ for completeness.)

Hence the number of ways to seat k students under these conditions is

$$p_{\leq n}(k) = \sum_{j=0}^n p(k, j).$$

There is no simpler closed-form expression for these partition numbers, but tables and generating functions can be used to compute them for specific k and n .

- (f) **We do not care who sits where, only which students sit *together in the same row* (no matter which row).**

In this case, the arrangement is determined solely by partitioning the set of k distinct students into nonempty subsets, where each subset corresponds to a group sitting together in one row (and the rows themselves are unlabeled). The number of ways to partition a k -element set into j nonempty subsets is given by the Stirling number of the second kind, denoted $S(k, j)$. Therefore, if we allow any number j of groups (with $1 \leq j \leq k$), the total number of ways is:

$$\sum_{j=1}^k S(k, j).$$

This sum is known as the Bell number B_k ; that is,

$$B_k = \sum_{j=1}^k S(k, j).$$

Thus, the number of ways is:

$$B_k.$$

- (f) We have a total of 10 toys:

$$\underbrace{C_1, C_2, C_3, C_4, C_5, C_6}_{6 \text{ distinct cars}} \quad \text{and} \quad \underbrace{D, D, D, D}_{4 \text{ identical dolls}},$$

which are to be placed into 10 identical boxes (i.e. unlabeled subsets). We want the number of different ways to partition these toys according to which cars appear together and how many dolls join them.

Step 1: Partition the Cars

First, partition the 6 *distinct* cars into k nonempty subsets (blocks). The number of ways to do this is given by the Stirling number of the second kind

$$S(6, k).$$

Since each of the 6 cars is distinct, we have $S(6, 1)$ ways to partition them into 1 block, $S(6, 2)$ ways into 2 blocks, ..., up to $S(6, 6)$ ways to put each car in its own block. Hence k ranges over $1 \leq k \leq 6$.

Step 2: Distribute the Dolls

Next, we need to distribute the 4 *identical* dolls among these k car-blocks *and possibly place some (or all) dolls in new subsets* that contain only dolls.

Let

a = the number of dolls placed into the k car-containing blocks,

so that the remaining $4 - a$ dolls may form additional subsets, each of which contains *only* dolls (with no cars).

Dolls Within the Existing k Blocks. Since the k blocks of cars are *distinct* (once the cars have been chosen to form those blocks), distributing a identical dolls among the k blocks is counted by the usual “stars and bars” formula:

$$\binom{a + k - 1}{k - 1}.$$

New Subsets of Only Dolls. Now, out of the $(4 - a)$ remaining dolls, we can create any number t of subsets ($0 \leq t \leq 4 - a$), each subset consisting of a positive number of dolls. Because these new subsets (boxes) are *unlabeled*, the number of ways to partition $(4 - a)$ identical dolls into exactly t positive parts is

$$p_t(4 - a),$$

where $p_t(n)$ denotes the number of ways to partition n identical items into t nonempty unlabeled subsets. Summing over all t from 0 to $(4 - a)$ then gives the total ways to form any number of doll-only subsets out of the $(4 - a)$ dolls.

Putting It All Together

For a fixed value of k (the number of car-blocks), the number of ways to handle the dolls is

$$\sum_{a=0}^4 \left[\binom{a + k - 1}{k - 1} \times \sum_{t=0}^{4-a} p_t(4 - a) \right].$$

We then multiply by $S(6, k)$, the ways to choose the k car-blocks in the first place. Finally, we sum over all possible k :

$$\text{Total ways} = \sum_{k=1}^6 \left[S(6, k) \times \sum_{a=0}^4 \left(\binom{a + k - 1}{k - 1} \times \sum_{t=0}^{4-a} p_t(4 - a) \right) \right].$$

A direct computation (by hand or program) yields the final count

$$\boxed{13,960}.$$

Hence, there are $\boxed{13,960}$ ways to place the 6 distinct cars and 4 identical dolls into 10 identical boxes.

- (g) **Everyone sits in the first 5 rows, and each of these 5 rows has the *same* number of students (assume $5 \mid k$).**

Let $k = 5m$. We have 5 labeled rows (rows 1 through 5), and each must have exactly m students. We do care about *which* students go to each row, and also which seats they occupy (and the students are distinct). The counting proceeds in two stages:

Preparation Tasks 1

- (a) Distribute the k students into 5 labeled groups of size m each:

$$\binom{k}{m} \binom{k-m}{m} \cdots = \frac{k!}{(m!)^5}.$$

- (b) For each group of m destined for row i , choose which m seats (out of n in that row) they occupy, and permute the m distinct students among those m seats:

$$\binom{n}{m} m! \quad \text{ways per row.}$$

Hence, for 5 rows, multiply $((\binom{n}{m} m!))^5$.

Overall, the number of ways is:

$$\frac{k!}{(m!)^5} \times ((\binom{n}{m} m!))^5 = k! \left[\binom{n}{m} \right]^5, \quad \text{where } m = \frac{k}{5}.$$

- (h) **Everyone sits in the first 3 rows; we do not care which physical seats are used, but we do care about left-right adjacency (ignoring empty chairs).**

Here each row is treated as a consecutive *line* of however many students it has. We do care which student is to the left or right of which other student in that row, but we ignore the actual seat numbers and any gaps. Thus, for row i having x_i students, the x_i chosen students can be arranged in $(x_i)!$ ways (linear order). Then we sum over all ways to split k distinct students into 3 labeled groups (for the 3 rows) and order each group:

- (a) Choose a triple (x_1, x_2, x_3) with $x_1 + x_2 + x_3 = k$.

- (b) Choose which x_1 students go to row 1, x_2 to row 2, etc. That is a multinomial factor $\frac{k!}{x_1! x_2! x_3!}$.

- (c) Permute each group internally in $(x_i)!$ ways.

But multiplying $\frac{k!}{x_1! x_2! x_3!} \times (x_1)!(x_2)!(x_3)! = k!$.

Finally, we sum over all nonnegative (x_1, x_2, x_3) summing to k . The number of such triples is $\binom{k+3-1}{3-1} = \binom{k+2}{2}$. Thus total seatings:

$$k! \binom{k+2}{2}.$$

- (i) **If a row is not empty, it contains *at least two* people; we do not distinguish students *and* we do not distinguish chairs in the same row.**

So we only care how many students are in each row, with $x_i = 0$ or $x_i \geq 2$. Since $k \leq n$, it is possible that many rows are empty. We want nonnegative $x_1, \dots, x_n$ with $x_1 + \cdots + x_n = k$ and each nonempty $x_i \geq 2$. Since we do *not* distinguish which particular students go to row i , nor which seats they occupy, we are effectively counting the integer solutions:

$$x_i \in \{0\} \cup \{2, 3, \dots\}, \quad x_1 + \cdots + x_n = k.$$

Let m be the number of nonempty rows. Then m satisfies $1 \leq m \leq \lfloor k/2 \rfloor$, and we choose which m rows are nonempty in $\binom{n}{m}$ ways. In each of those m chosen rows, let $x_i \geq 2$. Set $y_i = x_i - 2 \geq 0$, so $\sum_{i=1}^m y_i = k - 2m$. The number of nonnegative solutions to that is $\binom{(k-2m)+m-1}{m-1} = \binom{k-m-1}{m-1}$, valid as long as $k \geq 2m$.

Summing over all m gives

$$\sum_{m=1}^{\lfloor k/2 \rfloor} \binom{n}{m} \binom{k-m-1}{m-1}, \quad (\text{assuming } k > 0).$$

Preparation Tasks 2

(j) If a row is not empty, it has at least two people; we do not distinguish students, *but* we do care which specific seats are occupied.

(k) If a chair is empty, then all chairs *to its left in the same row* are empty as well.

Equivalently, in each row that has $x_i > 0$ seats occupied, those must be the leftmost x_i chairs of that row, with no gaps. If we do *not* distinguish students and only care about which seats are occupied, then row i is either completely empty ($x_i = 0$) or we occupy seats 1 through x_i . Summing over all rows, we must choose a total of k occupied chairs:

$$x_1 + x_2 + \cdots + x_n = k, \quad 0 \leq x_i \leq n.$$

Since $k \leq n$, the row capacity $x_i \leq n$ is never an issue. The number of nonnegative integer solutions is the standard stars-and-bars count:

$$\binom{k+n-1}{n-1}.$$

Thus there are $\binom{k+n-1}{n-1}$ ways to pick exactly which seats are occupied (each row's occupied block starts at seat 1).

6 Preparation Tasks 2

6.1 Problem 1

Prove that the num. of partitions of a positive int n into k *even* parts is eq. to the num. of partitions of n into k odd parts.

- **Idea:** First, express n in two ways by decomposing it into k even parts and into k odd parts. We can write

$$\begin{aligned} n &= 2a_1 + 2a_2 + \cdots + 2a_k, \\ n - k &= 2a_1 + 2a_2 + \cdots + 2a_k - k, \\ n - k &= (2a_1 - 1) + (2a_2 - 1) + \cdots + (2a_k - 1). \end{aligned}$$

The first line expresses n as a sum of k even numbers (each $2a_i$). Subtracting k from both sides shows that $n - k$ can be expressed as a sum of k odd numbers (each $2a_i - 1$).

From this idea, we expect a correspondence between partitions of n into k even parts and partitions of $n - k$ into k odd parts.

- **Formal proof:** Define the sets

$$P_e(n, k) := \{\text{partitions of } n \text{ into } k \text{ even parts}\},$$

and

$$P_o(n, k) := \{\text{partitions of } n \text{ into } k \text{ odd parts}\}.$$

For n even, it turns out these two sets have the same size:

$$|P_e(n, k)| = |P_o(n - k, k)|.$$

To show this, we construct an explicit bijection. Define a function

$$f : P_e(n, k) \rightarrow P_o(n - k, k),$$

by

$$f(b_1, b_2, \dots, b_k) = (b_1 - 1, b_2 - 1, \dots, b_k - 1).$$

Preparation Tasks 2

Here $(b_1, \dots, b_k) \in P_e(n, k)$ means $b_1 + \dots + b_k = n$ with each b_i even, so $b_i \geq 2$ for all i . Thus, each $b_i - 1 \geq 1$ and is odd, which ensures $f(b_1, \dots, b_k) \in P_o(n - k, k)$ is a partition of $n - k$ into k odd parts. The function f is well-defined. Moreover, it is invertible by simply adding 1 to each part. In fact, define

$$g : P_o(n - k, k) \rightarrow P_e(n, k)$$

as

$$g(c_1, c_2, \dots, c_k) = (c_1 + 1, c_2 + 1, \dots, c_k + 1).$$

If $(c_1, \dots, c_k)$ is a partition of $n - k$ into odd parts, then each c_i is odd and $c_i \geq 1$, so $c_i + 1$ is even and ≥ 2 , and $\sum_{i=1}^k (c_i + 1) = (n - k) + k = n$. Thus $g(c_1, \dots, c_k) \in P_e(n, k)$. It is easy to check that g is indeed the inverse of f : we have $f(g(c_1, \dots, c_k)) = (c_1, \dots, c_k)$ and $g(f(b_1, \dots, b_k)) = (b_1, \dots, b_k)$. Therefore, f is bijective.

6.2 Problem 2

Show that the number of partitions of a positive int n with at most k components is eq. to the num. of partitions of $2n$ with at most k even components.

- **Idea:** We consider partitions of n that have at most k parts. Let

$$n = a_1 + a_2 + \dots + a_L,$$

where $L \leq k$ and $a_1 \geq a_2 \geq \dots \geq a_L > 0$. (In other words, $a_1, \dots, a_L$ are the parts of a partition of n , listed in non-increasing order, with at most k parts.) For example, if $n = 5$ and $k = 3$, the partitions of 5 with at most 3 parts can be represented (padding with zeros up to 3 parts) as:

$$(2, 2, 1), \quad (3, 2, 0), \quad (5, 0, 0),$$

where we use 0 to indicate an empty part (no number in that position).

Notice that doubling each part in these examples produces a partition of $2n = 10$ with only even parts (and still at most 3 components). For instance, $(2, 2, 1)$ doubles to $(4, 4, 2)$, $(3, 2, 0)$ doubles to $(6, 4, 0)$, and $(5, 0, 0)$ doubles to $(10, 0, 0)$. This suggests a direct correspondence between partitions of n (up to k parts) and partitions of $2n$ into even parts (up to k parts).

- **Formal proof:** Let us define the relevant sets in words (as suggested in the notes):
 - $P(n, \leq k)$ — the set of all partitions of n with *at most* k components (parts).
 - $P_e(2n, \leq k)$ — the set of all partitions of $2n$ with at most k *even* components.

We aim to show that

$$|P(n, \leq k)| = |P_e(2n, \leq k)|,$$

i.e. the two sets have equal cardinality. To prove this, we construct a bijection $f : P(n, \leq k) \rightarrow P_e(2n, \leq k)$. Given any partition $(a_1, a_2, \dots, a_L)$ of n (with $L \leq k$), map it to

$$f(a_1, a_2, \dots, a_L) = (2a_1, 2a_2, \dots, 2a_L).$$

In other words, f doubles each part of the partition of n . If the partition of n has fewer than k parts, we may imagine that it is padded with zeros (as above) which double to zeros, so the resulting partition of $2n$ still has at most k parts. By construction, $f(a_1, \dots, a_L)$ is a partition of $2n$ in which every part is even, so indeed $f(a_1, \dots, a_L) \in P_e(2n, \leq k)$. The function f is invertible by halving each even part: for any partition $(b_1, b_2, \dots, b_M) \in P_e(2n, \leq k)$ (each b_i even), the inverse map f^{-1} gives

$$f^{-1}(b_1, b_2, \dots, b_M) = \left(\frac{b_1}{2}, \frac{b_2}{2}, \dots, \frac{b_M}{2} \right),$$

which is a partition of $2n/2 = n$ with at most k parts. Thus f is a bijection between $P(n, \leq k)$ and $P_e(2n, \leq k)$, and consequently $|P(n, \leq k)| = |P_e(2n, \leq k)|$.

7 Functions Between Sets

Let N and R be sets with $|N| = n$ and $|R| = r$.

- (i) **Total Functions:** The number of functions from N to R is

$$r^n.$$

Explanation: For every element in N , there are $|R| = r$ possible values in R . Thus, for the first element, there are r choices, for the second element, there are r choices, and so on. Applying the rule of product, the total number of functions is r^n .

- (ii) **Injective Functions:** When $r \geq n$, an injective function (one-to-one) from N to R can be chosen by assigning distinct images to the n elements.

If a function is injective, then for each value in the range there is only one corresponding argument. This means that function values cannot repeat, ensuring that $x_1 \neq x_2$ implies $f(x_1) \neq f(x_2)$.

Since there are $|R| = r$ choices for the first argument, $r - 1$ choices for the second, $r - 2$ for the third, and so on, applying the rule of product, the number of injective functions from N to R is:

$$r \cdot (r - 1) \cdots (r - n + 1) = \frac{r!}{(r - n)!}.$$

- (iii) **Surjective Functions:** A function is surjective (onto) if every element in R has a pre-image in N , meaning every element in R is an image of some element in N . Consider a surjection $f : N \rightarrow R = \{y_1, y_2, \dots, y_r\}$. We observe that the preimages $f^{-1}(y_1), f^{-1}(y_2), \dots, f^{-1}(y_r)$ form a partition of N into r non-empty subsets, as each element y_i in R corresponds to one or more elements from N . The number of ways to partition N into r parts is given by the Stirling number $S(n, r)$, and since we can permute the r elements in R in $r!$ ways, the total number of surjective functions from N to R is:

$$r! S(n, r),$$

where $S(n, r)$ is the Stirling number of the second kind, counting the ways to partition N into r non-empty subsets.

Example: For $N = \{1, 2, 3\}$ and $R = \{y_1, y_2\}$:

Here $|N| = 3$ and $|R| = 2$.

- Total functions: $2^3 = 8$.
- Injective functions: Not possible since $|R| < |N|$.
- Surjective functions: Consider all possible surjective functions:
 $f_1 : \{1, 2\} \mapsto y_1, 3 \mapsto y_2$ - Another possible permutation for this partition: $f_2 : \{1, 2\} \mapsto y_2, 3 \mapsto y_1$
 $f_3 : \{2, 3\} \mapsto y_1, 1 \mapsto y_2$ - Another possible permutation for this partition: $f_4 : 1 \mapsto y_1, \{2, 3\} \mapsto y_2$
 $f_5 : \{1, 3\} \mapsto y_1, 2 \mapsto y_2$ - Another possible permutation for this partition: $f_6 : 2 \mapsto y_1, \{1, 3\} \mapsto y_2$
 So, we have 6 surjective functions. Using the formula for surjective functions, we first find the Stirling number $S(3, 2) = 3$, which corresponds to the number of partitions without considering permutations. Then, accounting for the permutations of the $r = 2$ elements in R , we compute:

$$2! \cdot S(3, 2) = 2! \cdot 3 = 6,$$

which matches the number of surjective functions we listed.

8 Generating functions

8.1 Generating Series

Instead of viewing a sequence as a function that returns its n th term, a *generating series* packages all of its terms into a single power series whose coefficients are exactly the sequence entries. Concretely, the sequence

$$2, 3, 5, 8, 12, \dots$$

is encoded by the generating series

$$2 + 3x + 5x^2 + 8x^3 + 12x^4 + \dots$$

In general, given any sequence $\{c_n\}_{n \geq 0}$, its generating series is the formal power series

$$G(x) = \sum_{n=0}^{\infty} c_n x^n = c_0 + c_1 x + c_2 x^2 + c_3 x^3 + \dots$$

We say that $G(x)$ “generates” the sequence $\{c_n\}$ because each coefficient of x^n in $G(x)$ is precisely c_n . Generating series turn sequence-based problems into algebraic manipulations of power series, a technique we will exploit heavily in what follows.

Recall of the Basic Series

$a_0 = 1$	$a_1 = \frac{1}{2}$			
	$a_2 = \frac{1}{4}$	$a_3 = \frac{1}{8}$	$a_4 = \frac{1}{16}$	$\dots$

Figure 1: A geometric interpretation of the binary series, showing how $\sum_{n=0}^{\infty} \frac{1}{2^n} = 2$.

A Geometric View of the Binary Series For $|x| < 1$, we have the infinite geometric series

$$\frac{1}{1-x} = 1 + x + x^2 + x^3 + \dots = \sum_{n=0}^{\infty} x^n.$$

We now present a quick proof of this result by performing long division of 1 by $1-x$.

	$1 + x + x^2 + x^3 + \dots$
$1 - x$	1
	$1 - x$
	x
	$x - x^2$
	x^2
	$x^2 - x^3$
	x^3
	$\vdots$

The process works as follows: The long-division proceeds by repeatedly dividing the current remainder by the leading term of the divisor, producing one new power of x at each step:

1. Divide 1 by $1 - x$. The multiplier needed to eliminate the constant term is 1, so

$$1 - 1 \cdot (1 - x) = x.$$

Thus the first summand is 1, leaving a remainder of x .

2. Divide the remainder x by $1 - x$. The multiplier is x , so

$$x - x \cdot (1 - x) = x^2.$$

Hence the second summand is x , leaving a remainder of x^2 .

3. Divide x^2 by $1 - x$. The multiplier is x^2 , giving

$$x^2 - x^2 \cdot (1 - x) = x^3.$$

Therefore the third summand is x^2 , with remainder x^3 .

4. Continuing in this fashion produces the infinite series

$$\frac{1}{1-x} = 1 + x + x^2 + x^3 + \cdots.$$

Continuing indefinitely produces

$$\frac{1}{1-x} = 1 + x + x^2 + x^3 + \cdots = \sum_{n=0}^{\infty} x^n,$$

as claimed.

We will use this fact in further examples throughout the notes.

8.2 Building Generating Functions

The simplest (or “basic”) generating function is

$$\frac{1}{1-x} = 1 + x + x^2 + x^3 + \cdots,$$

which generates the constant sequence $1, 1, 1, \dots$

Replacing x with $-x$:

$$\frac{1}{1-(-x)} = \frac{1}{1+x} = 1 - x + x^2 - x^3 + \cdots,$$

generating $1, -1, 1, -1, \dots$

Replacing x with $3x$:

$$\frac{1}{1-3x} = 1 + 3x + 9x^2 + 27x^3 + \cdots,$$

generating $1, 3, 9, 27, \dots$

Scaling a sequence by 3:

$$\frac{3}{1-3x} = 3 + 9x + 27x^2 + 81x^3 + \cdots,$$

generating $3, 9, 27, 81, \dots$

Termwise addition of sequences:

Adding the generating functions for $1, 1, 1, \dots$ and $1, 3, 9, \dots$ gives

$$\frac{1}{1-x} + \frac{1}{1-3x} = 2 + 4x + 10x^2 + 28x^3 + \dots,$$

which generates $2, 4, 10, 28, \dots$

Replacing x with x^2 :

$$\frac{1}{1-x^2} = 1 + x^2 + x^4 + x^6 + \dots,$$

generating $1, 0, 1, 0, 1, 0, \dots$

Shifting a sequence:

Multiplying by x shifts all coefficients right by one:

$$\frac{x}{1-3x} = 0 + x + 3x^2 + 9x^3 + \dots,$$

generating $0, 1, 3, 9, \dots$, and

$$\frac{x}{1-x^2} = 0 + x + 0x^2 + x^3 + \dots,$$

generating $0, 1, 0, 1, \dots$

Combining shifted sequences:

Adding the two “even-odd” generating functions recovers

$$\frac{1}{1-x^2} + \frac{x}{1-3x} = \frac{1+x}{1-x^2} = \frac{1}{1-x},$$

which generates $1, 1, 1, 1, \dots$

Differentiation:

Differentiating the basic Generating Function

$$\frac{d}{dx} \left(\frac{1}{1-x} \right) = \frac{1}{(1-x)^2} = 1 + 2x + 3x^2 + 4x^3 + \dots,$$

yields the generating function for $1, 2, 3, 4, \dots$

8.3 Recurrence Relations & Generating Functions

We conclude with an example of one of the many reasons studying generating functions is helpful: solving recurrence relations via algebraic manipulation of power series.

Example: Tower of Hanoi The minimum number of moves required to transfer n disks satisfies

$$a_0 = 0, \quad a_1 = 1, \quad a_n = 2a_{n-1} + 1 \quad (n \geq 1),$$

giving the sequence

$$0, 1, 3, 7, 15, 31, \dots$$

Define the generating function

$$f(x) = \sum_{n=0}^{\infty} a_n x^n.$$

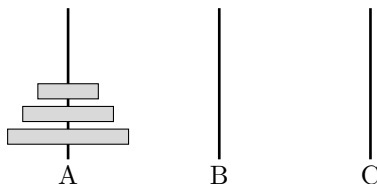


Figure 2: Initial configuration for Tower of Hanoi (3 disks).

Using the recurrence for $n \geq 1$:

$$\sum_{n=1}^{\infty} a_n x^n = \sum_{n=1}^{\infty} (2a_{n-1} + 1)x^n = 2x \sum_{n=0}^{\infty} a_n x^n + \sum_{n=1}^{\infty} x^n,$$

so

$$f(x) - a_0 = 2x f(x) + \frac{x}{1-x},$$

and since $a_0 = 0$,

$$f(x) = \frac{x}{(1-x)(1-2x)}.$$

Performing partial fractions:

$$\frac{x}{(1-x)(1-2x)} = \frac{-1}{1-x} + \frac{1}{1-2x},$$

hence

$$f(x) = -\frac{1}{1-x} + \frac{1}{1-2x}.$$

Extracting coefficients yields the closed-form solution

$$a_n = 2^n - 1,$$

confirming the well-known formula for the Tower of Hanoi moves.

8.4 Introduction to the Fibonacci Sequence

The Fibonacci sequence famously arises from a puzzle involving rabbit populations. Imagine starting with a single pair of rabbits that takes one month to mature. After maturing, each pair produces a new pair of rabbits every month. Mathematically, if F_n represents the number of rabbit pairs in month n , the sequence satisfies the initial conditions

$$F_0 = 0, \quad F_1 = 1,$$

and the recurrence

$$F_{n+2} = F_{n+1} + F_n \quad \text{for } n \geq 0.$$

Q: is there a non-recursive (closed-form) formula for F_n ?

Idea: consider and calculate it

- 1) $\parallel$ (2 small rabbits)
- 2) $\parallel + \parallel$ (1 big pair + 1 small pair)
- 3) $\parallel + \parallel + \parallel$ (2 big pairs + 1 small pair)
- 4) $\parallel + \parallel + \parallel + \parallel + \parallel$ (3 big pairs + 2 small pairs)

Figure 3: Illustration of rabbit pairs over successive months. Blue bars represent small rabbits; orange bars represent big (mature) rabbits.

8.5 Deriving the Closed-Form for the Fibonacci Sequence

Step 1: Define the generating function. Let $\{F_n\}_{n=0}^{\infty}$ be the Fibonacci sequence with

$$F_0 = 0, \quad F_1 = 1, \quad F_{n+2} = F_{n+1} + F_n \quad (n \geq 0).$$

Define the generating function

$$f(x) = \sum_{n=0}^{\infty} F_n x^n.$$

We aim to find a closed-form expression for $f(x)$, and then extract a formula for F_n .

Step 2: Use the Fibonacci recurrence in $f(x)$. Starting from

$$f(x) = F_0 + F_1 x + \sum_{n=2}^{\infty} F_n x^n,$$

and noting $F_0 = 0$, $F_1 = 1$, we have

$$f(x) = x + \sum_{n=2}^{\infty} (F_{n-1} + F_{n-2}) x^n$$

because $F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$. Separate the sums:

$$f(x) = x + \sum_{n=2}^{\infty} F_{n-1} x^n + \sum_{n=2}^{\infty} F_{n-2} x^n.$$

Shift indices to factor out $f(x)$:

$$\sum_{n=2}^{\infty} F_{n-1} x^n = x \sum_{n=2}^{\infty} F_{n-1} x^{n-1} = x \sum_{m=1}^{\infty} F_m x^m = x(f(x) - F_0) = x f(x),$$

since $F_0 = 0$. Similarly,

$$\sum_{n=2}^{\infty} F_{n-2} x^n = x^2 \sum_{n=2}^{\infty} F_{n-2} x^{n-2} = x^2 \sum_{k=0}^{\infty} F_k x^k = x^2 f(x).$$

Hence,

$$f(x) = x + x f(x) + x^2 f(x) \implies f(x)(1 - x - x^2) = x.$$

Thus,

$$f(x) = \frac{x}{1 - x - x^2}.$$

Step 3: Partial-Fraction Decomposition (as in the images). First, rewrite

$$\frac{1}{1-x-x^2} = \frac{1}{-(x^2+x-1)} = -\frac{1}{x^2+x-1}.$$

Next, factor $x^2 + x - 1$. Observe that the roots of

$$x^2 + x - 1 = 0$$

are

$$x = -\frac{1+\sqrt{5}}{2} \quad \text{and} \quad x = -\frac{1-\sqrt{5}}{2}.$$

Hence,

$$x^2 + x - 1 = \left(x + \frac{1+\sqrt{5}}{2}\right) \cdot \left(x + \frac{1-\sqrt{5}}{2}\right).$$

Therefore,

$$-\frac{1}{x^2+x-1} = -\frac{1}{\left(x + \frac{1+\sqrt{5}}{2}\right)\left(x + \frac{1-\sqrt{5}}{2}\right)}.$$

We look for constants A and B such that

$$-\frac{1}{\left(x + \frac{1+\sqrt{5}}{2}\right)\left(x + \frac{1-\sqrt{5}}{2}\right)} = \frac{A}{x + \frac{1+\sqrt{5}}{2}} + \frac{B}{x + \frac{1-\sqrt{5}}{2}}.$$

Step 4: Solve for A and B . Comparing coefficients of x and the constant term in

$$-1 = A\left(x + \frac{1+\sqrt{5}}{2}\right) + B\left(x + \frac{1-\sqrt{5}}{2}\right),$$

we obtain the system

$$\begin{cases} A + B = 0, \\ A\beta + B\alpha = -1. \end{cases}$$

It follows that

$$B = -A, \quad A(\beta - \alpha) = -1 \implies A = \frac{1}{\alpha - \beta} \quad \text{and} \quad B = -\frac{1}{\alpha - \beta}.$$

Hence,

$$-\frac{1}{(x+\alpha)(x+\beta)} = \frac{1}{\alpha-\beta} \frac{1}{x+\alpha} - \frac{1}{\alpha-\beta} \frac{1}{x+\beta}.$$

Step 5: Combine with the earlier factor -1 and rewrite. Recalling that

$$\frac{1}{1-x-x^2} = -\frac{1}{x^2+x-1} = -\frac{1}{(x+\alpha)(x+\beta)},$$

we combine the above result to conclude

$$\frac{1}{1-x-x^2} = \frac{1}{\alpha-\beta} \left(\frac{1}{x+\alpha} - \frac{1}{x+\beta} \right).$$

Step 6: Expand each term in a power series. Notice that

$$\frac{1}{x + \alpha} = \frac{1}{\alpha} \frac{1}{1 + \frac{x}{\alpha}} = \frac{1}{\alpha} \sum_{n=0}^{\infty} \left(-\frac{x}{\alpha}\right)^n = \sum_{n=0}^{\infty} \frac{(-1)^n}{\alpha^{n+1}} x^n,$$

valid for $|\frac{x}{\alpha}| < 1$. Similarly,

$$\frac{1}{x + \beta} = \sum_{n=0}^{\infty} \frac{(-1)^n}{\beta^{n+1}} x^n.$$

Hence,

$$\frac{1}{1 - x - x^2} = \frac{1}{\alpha - \beta} \left[\sum_{n=0}^{\infty} \frac{(-1)^n}{\alpha^{n+1}} x^n - \sum_{n=0}^{\infty} \frac{(-1)^n}{\beta^{n+1}} x^n \right] = \sum_{n=0}^{\infty} \left[\frac{1}{\alpha - \beta} \left(\frac{(-1)^n}{\alpha^{n+1}} - \frac{(-1)^n}{\beta^{n+1}} \right) \right] x^n.$$

Step 7: Identify Fibonacci numbers. Recall that $\alpha - \beta = \sqrt{5}$, and

$$F_n = \frac{\alpha^n - \beta^n}{\alpha - \beta} = \frac{\alpha^n - \beta^n}{\sqrt{5}}.$$

One checks (or uses known identities) to see that the coefficient of x^n in the above power series is exactly F_n . Consequently,

$$\sum_{n=0}^{\infty} F_n x^n = \frac{1}{1 - x - x^2},$$

which is the generating function for the Fibonacci sequence.

Conclusion. We have shown that the generating function for the Fibonacci sequence is $\frac{x}{1-x-x^2}$. Through partial fractions and comparing coefficients, we deduced that

$$F_n = \frac{\alpha^n - \beta^n}{\sqrt{5}}.$$

This gives a non-recursive (closed-form) expression for F_n , completing the derivation.

8.6 More examples

In earlier sections (see, e.g., *A Geometric View of the Binary Series* on page 14), we explored methods to solve recurrences and introduced generating functions as a tool to transform sequences into functions. In this section, we briefly reiterate these ideas and demonstrate, through several examples, how generating functions serve as a bridge between discrete mathematics and calculus.

Example 1: Constant Sequence. Consider the sequence defined by

$$a_n = 1 \quad \text{for all } n \geq 0,$$

so that the sequence is

$$1, 1, 1, \dots$$

By the geometric series formula (proved earlier), its generating function is given by

$$f(x) = \sum_{n=0}^{\infty} x^n = \frac{1}{1-x}, \quad |x| < 1.$$

Example 2: Exponential Sequence. Now, let

$$a_n = \frac{1}{n!}.$$

Then the generating function is

$$f(x) = \sum_{n=0}^{\infty} \frac{x^n}{n!} = e^x, \quad x \in \mathbb{R}.$$

A partial justification of this result can be obtained by recalling the Taylor series expansion of the exponential function. Although a complete treatment of Taylor series is a topic in calculus (not yet covered in this course), note that differentiating the power series term-by-term confirms the identity.

Example 3: Binomial Coefficient Sequence. Consider the sequence defined by

$$a_n = \binom{n+k}{k}.$$

Theorem. The generating function for this sequence is

$$f(x) = \sum_{n=0}^{\infty} \binom{n+k}{k} x^n = \frac{1}{(1-x)^{k+1}}, \quad |x| < 1.$$

Proof.

- For $k = 1$: Note that

$$a_n = \binom{n+1}{1} = n+1,$$

so that

$$f(x) = \sum_{n=0}^{\infty} \binom{n+1}{1} x^n = \sum_{n=0}^{\infty} (n+1)x^n.$$

Recall the geometric series,

$$\frac{1}{1-x} = \sum_{n=0}^{\infty} x^n,$$

and observe that by differentiating both sides term-by-term with respect to x , we can derive the generating function for the sequence $(n+1)$. In detail, differentiate the left-hand side:

$$\frac{d}{dx} \left(\frac{1}{1-x} \right) = \frac{1}{(1-x)^2}.$$

On the right-hand side, notice that since

$$\frac{d}{dx} x^{n+1} = (n+1)x^n,$$

differentiating the series yields

$$\frac{d}{dx} \left(\sum_{n=0}^{\infty} x^n \right) = \sum_{n=0}^{\infty} (n+1)x^n.$$

Thus, we conclude that

$$\frac{1}{(1-x)^2} = \sum_{n=0}^{\infty} (n+1)x^n.$$

This recovers the generating function for $k = 1$. A less formal derivation was given in the subsection *Building Generating Functions* on page 16.

A complete inductive proof follows similar lines but is omitted here for brevity.

Example 4: Alternating Factorial Sequence. Define the sequence by

$$a_n = \begin{cases} 0, & \text{if } n \text{ is even,} \\ \frac{(-1)^{\frac{n-1}{2}}}{n!}, & \text{if } n \text{ is odd.} \end{cases}$$

Then the generating function is

$$f(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots = \sin(x), \quad x \in \mathbb{R}.$$

Even though a full treatment of the Taylor series for trigonometric functions is part of calculus (again, a topic not yet covered here), this example illustrates how generating functions capture nontrivial sequence behavior by connecting discrete structures with analytic functions.

8.7 Generating Function Applications

One key application is the multiplication (or convolution) of generating functions, which naturally arises when we combine two distinct combinatorial constructions into a single, more complex structure.

Question: If a_k counts all objects of type A of size k and b_k counts all objects of type B of size k , how many pairs of objects (A, B) have a total size of n ?

Answer: The number of such pairs is given by

$$\sum_{k=0}^n a_k b_{n-k}.$$

Observation: The generating function for the sequence

$$C_n = \sum_{k=0}^n a_k b_{n-k}$$

is

$$\sum_{n=0}^{\infty} C_n x^n = \sum_{n=0}^{\infty} \left(\sum_{k=0}^n a_k b_{n-k} \right) x^n.$$

It shows that multiplying the generating functions corresponding to $\{a_n\}$ and $\{b_n\}$ produces a new generating function whose coefficients are given by the convolution of the two original sequences.

Example 1: Dice Sum Counting. A classic example of this application is counting the number of ways to obtain a given sum when rolling two standard six-sided dice. For a single die, the generating function is:

$$D(x) = x + x^2 + x^3 + x^4 + x^5 + x^6,$$

where the term x^k corresponds to rolling a k . Since the two dice are independent, the generating function for the sum of the two dice is:

$$D(x)^2 = (x + x^2 + x^3 + x^4 + x^5 + x^6)^2.$$

Expanding this product, the coefficient of x^n in $D(x)^2$ equals the number of ways to achieve a total sum of n . For instance, one can verify that the coefficient of x^7 is 6, which corresponds to the six possible outcomes that sum to 7 (namely, the pairs $(1, 6), (2, 5), (3, 4), (4, 3), (5, 2), (6, 1)$).

Example 2: Candy Selection Problem. Selecting 30 candies from 20 large types (each type can be picked at most once) and 40 small types (each type can be picked in any quantity) can be modeled with generating functions. For a single large candy type (available at most once), the generating function is

$$1 + x,$$

and for 20 independent large types, the combined generating function is

$$(1 + x)^{20}.$$

For a small candy type (with unlimited supply), the generating function is

$$1 + x + x^2 + \cdots = \frac{1}{1 - x},$$

so for 40 small types it is

$$\left(\frac{1}{1 - x} \right)^{40} = (1 - x)^{-40}.$$

Thus, the overall generating function becomes

$$G(x) = (1+x)^{20} (1-x)^{-40}.$$

To determine the number of ways to select 30 candies, we need the coefficient of x^{30} in $G(x)$. Expanding,

$$(1+x)^{20} = \sum_{i=0}^{20} \binom{20}{i} x^i, \quad (1-x)^{-40} = \sum_{j \geq 0} \binom{39+j}{39} x^j,$$

the convolution gives:

$$[x^{30}] G(x) = \sum_{i=0}^{20} \binom{20}{i} \binom{39+30-i}{39}.$$

Using a Vandermonde's convolution argument, one can show that

$$\sum_{i=0}^{20} \binom{20}{i} \binom{69-i}{39} = \binom{59}{30}.$$

Example 3: Selection with Limited Green Items. Consider selecting 20 objects from three categories:

1. An infinite pile of red objects,
2. An infinite pile of blue objects,
3. A pile of green objects with only 5 available.

For the red and blue objects (with unlimited supply), the generating function is:

$$\frac{1}{1-x},$$

so for both together we have:

$$\frac{1}{(1-x)^2}.$$

For the green objects (at most 5), the generating function is:

$$1 + x + x^2 + x^3 + x^4 + x^5 = \frac{1-x^6}{1-x}.$$

Thus, the overall generating function becomes:

$$G(x) = \frac{1}{(1-x)^2} \cdot \frac{1-x^6}{1-x} = \frac{1-x^6}{(1-x)^3}.$$

Using the expansion

$$(1-x)^{-3} = \sum_{n \geq 0} \binom{n+2}{2} x^n,$$

the coefficient of x^{20} in $G(x)$ is computed by writing:

$$G(x) = (1-x)^{-3} - x^6(1-x)^{-3}.$$

The coefficient from the first term is $\binom{22}{2} = 231$ (since $[x^{20}](1-x)^{-3} = \binom{20+2}{2}$) and from the second term, it is $\binom{16}{2} = 120$ (as the x^6 shifts the index, so $[x^{20}](x^6(1-x)^{-3}) = [x^{14}](1-x)^{-3}$). Hence,

$$[x^{20}] G(x) = 231 - 120 = 111.$$

Example 4: Coin Change with Limited Denominations. Determine the number of ways to make 10 (units) using coins of value 1, 2, and 5, where 1- and 2-unit coins are available in unlimited supply but 5-unit coins are limited to at most 2. The generating functions are:

$$G_1(x) = \frac{1}{1-x} \quad (\text{for 1-unit coins}),$$

$$G_2(x) = \frac{1}{1-x^2} \quad (\text{for 2-unit coins}),$$

$$G_5(x) = 1 + x^5 + x^{10} \quad (\text{for 5-unit coins, at most 2}).$$

Thus, the overall generating function is:

$$G(x) = \frac{1}{(1-x)(1-x^2)} (1 + x^5 + x^{10}).$$

To find the coefficient of x^{10} , write:

$$G(x) = \frac{1}{(1-x)(1-x^2)} + \frac{x^5}{(1-x)(1-x^2)} + \frac{x^{10}}{(1-x)(1-x^2)}.$$

The first term contributes the number of ways to form 10 with 1- and 2-unit coins, which is 6; the second term contributes the number of ways to form 5 (which is 3); and the third term contributes 1 (making 0 with 1- and 2-unit coins). Therefore, the total number of ways is:

$$6 + 3 + 1 = 10.$$

Example 5: Composition with Constrained Part Sizes. Count the number of compositions of 7 into exactly 3 positive parts, where each part is at most 4. The generating function for a single part that can take values 1 through 4 is:

$$F(x) = x + x^2 + x^3 + x^4.$$

For a composition with exactly 3 parts, the generating function is:

$$G(x) = [F(x)]^3 = (x + x^2 + x^3 + x^4)^3.$$

A term x^7 in the expansion corresponds to a composition of 7 into 3 parts. Without the upper bound, the number of compositions of 7 into 3 parts (with each part at least 1) is given by

$$\binom{7-1}{3-1} = \binom{6}{2} = 15.$$

However, we must exclude compositions where any part exceeds 4. In this specific case, the only forbidden compositions are those where one part is 5 and the other two are 1 (i.e., $5 + 1 + 1$ and its permutations), which count to 3. Hence, the number of valid compositions is:

$$15 - 3 = 12.$$

9 Generating functions tutorial tasks

Preliminaries: two work-horse identities

Throughout the solutions we repeatedly invoke the following elementary—but indispensable—summation formulas. Which have already been discussed on the lecture. They are stated once here for quick reference and are *not* re-derived every time we use them.

Infinite geometric series

$$\sum_{n=0}^{\infty} r^n = \frac{1}{1-r}, \quad |r| < 1. \quad (1)$$

Finite geometric series

$$\sum_{n=0}^N r^n = \frac{1-r^{N+1}}{1-r}, \quad r \neq 1, \quad N \in \mathbb{N}. \quad (2)$$

Tasks:

- Find generating functions $G(x) = \sum_{n=0}^{\infty} a_n x^n$ for each of the following sequences:
 - $a_n = \alpha^n, \quad n = 0, 1, 2, \dots, \alpha \in \mathbb{R}.$
 - $a_n = \begin{cases} 1, & n = 0, 1, \dots, N, \\ 0, & n > N. \end{cases}$
 - $a_n = \begin{cases} n+1, & n = 0, 1, \dots, N, \\ 0, & n > N. \end{cases}$
 - $a_n = \alpha n, \quad n = 0, 1, 2, \dots, \alpha \in \mathbb{R}.$
 - $a_n = n^2, \quad n = 0, 1, 2, \dots$
 - $a_n = n \alpha^n, \quad n = 0, 1, 2, \dots, \alpha \in \mathbb{R}.$
- Let $f(x) = \sum_{n=0}^{\infty} a_n x^n$ be the ordinary generating function of the sequence $(a_n)_{n \geq 0}$. For each definition of A_n below, determine the generating function $F(x) = \sum_{n=0}^{\infty} A_n x^n$.
 - $A_n = a_{n+1}.$
 - $A_n = a_{n+k}, \quad \text{fixed } k \in \mathbb{N}.$
 - $A_n = a_{n+1} - a_n.$
 - $A_n = n a_n.$
 - $A_n = \begin{cases} a_{n-1}, & n \geq 1, \\ 0, & n = 0. \end{cases}$

Solution & Derivations

1. Generating functions of the given sequences

We always write $G(x) = \sum_{n \geq 0} a_n x^n$.

- 1(a) $a_n = \alpha^n.$

Start by writing the series explicitly:

$$f(x) = 1 + \alpha x + (\alpha x)^2 + (\alpha x)^3 + \dots$$

Now compress the same string of terms into $\sum$ -notation:

$$f(x) = \sum_{n=0}^{\infty} (\alpha x)^n, \quad |\alpha x| < 1.$$

Since this is a geometric series with ratio $r = \alpha x$ (and $|r| < 1$ for convergence), we invoke the closed form

$$\sum_{n=0}^{\infty} r^n = \frac{1}{1-r}.$$

Substituting $r = \alpha x$ gives

$$G(x) = \frac{1}{1-\alpha x}.$$

1(b) Truncated arithmetic sequence $a_n = n + 1$.

$$\begin{aligned} f(x) &= \sum_{n=0}^N (n+1)x^n = \\ &= \sum_{n=0}^N \frac{d}{dx} x^{n+1} = \\ &= \frac{d}{dx} \sum_{n=0}^N x^{n+1} = \\ &= \frac{d}{dx} \left(x \frac{1-x^{N+1}}{1-x} \right) = \\ &= \frac{1 - (N+2)x^{N+1} + (N+1)x^{N+2}}{(1-x)^2}. \end{aligned}$$

1(c) $a_n = \alpha n$.

$$f(x) = 0 + \alpha x + 2\alpha x^2 + 3\alpha x^3 + \dots$$

$$f(x) = \sum_{n=0}^{\infty} \alpha n x^n, \quad |x| < 1.$$

$$\sum_{n=0}^{\infty} n x^n = x \frac{d}{dx} \sum_{n=0}^{\infty} x^n = x \frac{d}{dx} \left(\frac{1}{1-x} \right) = \frac{x}{(1-x)^2},$$

$$f(x) = \alpha \frac{x}{(1-x)^2}.$$

1(d) $a_n = n^2$.

Define the basic geometric generating function

$$F(x) = \sum_{n=0}^{\infty} x^n = \frac{1}{1-x}.$$

Then view each higher-order sum as a simple transformation of F :

$$F_1(x) = x F'(x) = \sum_{n=0}^{\infty} n x^n = \frac{x}{(1-x)^2},$$

$$F_2(x) = x \frac{d}{dx} F_1(x) = \sum_{n=0}^{\infty} n^2 x^n = x \frac{d}{dx} \left(\frac{x}{(1-x)^2} \right) = \frac{x(1+x)}{(1-x)^3}.$$

Thus

$$\boxed{\sum_{n=0}^{\infty} n^2 x^n = F_2(x) = \frac{x(1+x)}{(1-x)^3}, \quad |x| < 1.}$$

1(e) $a_n = n \alpha^n.$

$$G(x) = 0 + \alpha x + 2\alpha^2 x^2 + 3\alpha^3 x^3 + \dots$$

$$G(x) = \sum_{n=0}^{\infty} n \alpha^n x^n = \sum_{n=0}^{\infty} n (\alpha x)^n, \quad |\alpha x| < 1.$$

$$\begin{aligned} G(x) &= x \frac{d}{dx} \sum_{n=0}^{\infty} (\alpha x)^n = x \frac{d}{dx} \left(\frac{1}{1-\alpha x} \right) \\ &= x \cdot \frac{\alpha}{(1-\alpha x)^2} = \frac{\alpha x}{(1-\alpha x)^2}. \end{aligned}$$

2. From $f(x)$ to $F(x)$: shifts, differences, multipliers

Let $f(x) = \sum_{n \geq 0} a_n x^n$ be *given*. Each transformation of the indices has a mechanical counterpart on the generating function—think of these as “operator moves” on $f(x)$. No magic, just algebra.

2(a) **Shift forward by one index:** $A_n = a_{n+1}.$

$$F(x) = \sum_{n \geq 0} a_{n+1} x^n$$

Rewriting the sum as:

$$\sum_{n=1}^{\infty} a_n x^{n-1} = x \cdot \left(\sum_{n=1}^{\infty} a_n x^n \right) = x \cdot \left(\sum_{n=0}^{\infty} a_n x^n - a_0 \right) = \frac{1}{x} (f(x) - a_0)$$

2(b) **Shift forward by k :** $A_n = a_{n+k}.$

$$F(x) = \sum_{n=0}^{\infty} a_{n+k} x^n$$

Step 1: Rewriting the sum with a change in the index:

$$F(x) = \sum_{n=k}^{\infty} a_n x^{n-k}$$

Step 2: Factor out x^k :

$$F(x) = x^{-k} \sum_{n=k}^{\infty} a_n x^n$$

Step 3: Final expression using $f(x)$:

Recall that:

$$f(x) = \sum_{n=0}^{\infty} a_n x^n$$

Thus, we can express $F(x)$ as:

$$F(x) = x^{-k} \left(f(x) - \sum_{n=0}^{k-1} a_n x^n \right)$$

2(c) First difference: $A_n = a_{n+1} - a_n$.

$$F(x) = \sum_{n \geq 0} (a_{n+1} - a_n) x^n$$

Step 1: Express $f(x)$ and subtract off a_0 :

$$f(x) = \sum_{n=0}^{\infty} a_n x^n$$

$$f(x) - a_0 = \sum_{n=1}^{\infty} a_n x^n$$

Thus, we can rewrite the sum as:

$$F(x) = [f(x) - a_0] \frac{1}{x} - f(x)$$

Step 2: Factor and simplify:

$$F(x) = \frac{f(x)(1-x) - a_0}{x}$$

2(d) Index-multiplier: $A_n = n a_n$.

$$F(x) = \sum_{n \geq 0} n \cdot a_n x^n = \sum_{n=1}^{\infty} n \cdot a_n x^{n-1} \cdot x = x f'(x)$$

2(e) **Lag operator:** $A_n = a_{n-1}$ for $n \geq 1$, $A_0 = 0$.

Here we simply *multiply* by x :

$$F(x) = x \sum_{m \geq 0} a_m x^m = x f(x).$$

Bottom line. Generating functions live and die by (1)–(2) plus elementary calculus. Once you stop being sentimental about the indices and treat x as an operator knob, everything else collapses to high-school algebra. Don't over-complicate it.

Recurrence-Relation Exercises

Use generating functions to find a closed form for each of the following sequences:

(a) $a_n = 6n + a_{n-1}$, $n \geq 1$, $a_0 = 0$.

(b) $a_{n+2} = 2a_{n+1} + 3a_n$, $n \geq 0$, $a_0 = 1$, $a_1 = 2$.

(c) $a_n = -a_{n-1} + 2a_{n-2}$, $n \geq 2$, $a_0 = 1$, $a_1 = 2$.

Solution to (a)

Define the ordinary generating function

$$f(x) = \sum_{n=0}^{\infty} a_n x^n.$$

Multiply the recurrence $a_n = 6n + a_{n-1}$ by x^n and sum for $n \geq 1$:

$$\sum_{n=1}^{\infty} a_n x^n = \sum_{n=1}^{\infty} a_{n-1} x^n + 6 \sum_{n=1}^{\infty} n x^n \implies f(x) - a_0 = x f(x) + 6 \sum_{n=1}^{\infty} n x^n.$$

We compute the weighted sum by differentiation:

$$\sum_{n=1}^{\infty} n x^n = x \frac{d}{dx} \left(\sum_{n=0}^{\infty} x^n \right) = x \frac{d}{dx} \left(\frac{1}{1-x} \right) = \frac{x}{(1-x)^2}.$$

Since $a_0 = 0$, the functional equation becomes

$$f(x)(1-x) = 6 \frac{x}{(1-x)^2} \implies f(x) = \frac{6x}{(1-x)^3}.$$

$$(1-x)^{-\alpha} = \sum_{k=0}^{\infty} \binom{\alpha+k-1}{k} x^k \quad (\alpha \in \mathbb{N}, \text{ as covered in lectures and tutorials})$$

In particular, for $\alpha = 3$ this gives $(1-x)^{-3} = \sum_{k=0}^{\infty} \binom{k+2}{2} x^k$.

Finally, substituting into $f(x)$ yields

$$f(x) = 6x \sum_{k=0}^{\infty} \binom{k+2}{2} x^k = 6 \sum_{n=1}^{\infty} \binom{n+1}{2} x^n \implies a_n = 6 \binom{n+1}{2} = 3n(n+1).$$

Solution to (b)

Define the ordinary generating function

$$f(x) = \sum_{n=0}^{\infty} a_n x^n.$$

Multiply the recurrence $a_{n+2} = 2a_{n+1} + 3a_n$ by x^n and sum for $n \geq 0$:

$$\begin{aligned} \sum_{n=0}^{\infty} a_{n+2} x^n &= \sum_{m=2}^{\infty} a_m x^{m-2} && (\text{set } m = n + 2) \\ x^{-2} \sum_{m=2}^{\infty} a_m x^m &= x^{-2} (f(x) - a_0 - a_1 x), \\ \sum_{n=0}^{\infty} a_{n+1} x^n &= \sum_{m=1}^{\infty} a_m x^{m-1} = x^{-1} (f(x) - a_0), \\ \sum_{n=0}^{\infty} a_n x^n &= f(x). \end{aligned}$$

Putting these into $\sum a_{n+2} x^n = 2 \sum a_{n+1} x^n + 3 \sum a_n x^n$ gives

$$x^{-2} (f(x) - a_0 - a_1 x) = 2x^{-1} (f(x) - a_0) + 3f(x).$$

Substitute $a_0 = 1$, $a_1 = 2$ and clear denominators:

$$f(x) - 1 - 2x = 2x(f(x) - 1) + 3x^2 f(x) \implies f(x)(1 - 2x - 3x^2) = 1 \implies f(x) = \frac{1}{(1+x)(1-3x)}.$$

$$(1-r)^{-1} = \sum_{n=0}^{\infty} r^n \quad (\text{as covered in lectures and tutorials})$$

Use partial fractions:

$$\frac{1}{(1+x)(1-3x)} = \frac{\frac{1}{4}}{1+x} + \frac{\frac{3}{4}}{1-3x},$$

and expand each by the boxed identity with $r = -x$ and $r = 3x$:

$$f(x) = \frac{1}{4} \sum_{n=0}^{\infty} (-x)^n + \frac{3}{4} \sum_{n=0}^{\infty} (3x)^n = \sum_{n=0}^{\infty} \left(\frac{1}{4} (-1)^n + \frac{3}{4} 3^n \right) x^n.$$

Hence

$$a_n = \frac{1}{4} (-1)^n + \frac{3}{4} 3^n.$$

Solution to (c)

Define the ordinary generating function

$$f(x) = \sum_{n=0}^{\infty} a_n x^n.$$

Generating functions tutorial tasks

Multiply the recurrence $a_n = -a_{n-1} + 2a_{n-2}$ by x^n and sum for $n \geq 2$:

$$\begin{aligned}\sum_{n=2}^{\infty} a_n x^n &= -\sum_{n=2}^{\infty} a_{n-1} x^n + 2\sum_{n=2}^{\infty} a_{n-2} x^n \\ &= -x \sum_{m=1}^{\infty} a_m x^m + 2x^2 \sum_{m=0}^{\infty} a_m x^m \\ &= -x(f(x) - a_0) + 2x^2 f(x),\end{aligned}$$

while

$$\sum_{n=2}^{\infty} a_n x^n = f(x) - a_0 - a_1 x.$$

Substituting $a_0 = 1$, $a_1 = 2$ yields

$$f(x) - 1 - 2x = -x(f(x) - 1) + 2x^2 f(x) \implies f(x)(1 + x - 2x^2) = 1 + 3x \implies f(x) = \frac{1 + 3x}{(1 - x)(1 + 2x)}.$$

Decompose into partial fractions:

$$\frac{1 + 3x}{(1 - x)(1 + 2x)} = \frac{A}{1 - x} + \frac{B}{1 + 2x} \implies A = \frac{4}{3}, B = -\frac{1}{3},$$

so

$$f(x) = \frac{4/3}{1 - x} - \frac{1/3}{1 + 2x}.$$

Use the geometric-series identity

$$\frac{1}{1 - r} = \sum_{n=0}^{\infty} r^n \quad (\text{as in lectures and tutorials})$$

with $r = x$ and $r = -2x$ to get

$$f(x) = \frac{4}{3} \sum_{n=0}^{\infty} x^n - \frac{1}{3} \sum_{n=0}^{\infty} (-2x)^n = \sum_{n=0}^{\infty} \left(\frac{4}{3} - \frac{(-2)^n}{3} \right) x^n.$$

Therefore

$$a_n = \frac{4 - (-2)^n}{3}.$$

Tasks: Coefficient Extraction

Use generating-function techniques to find the coefficient of x^{12} in each of the following:

(a) $(1 + x^3 + x^6 + x^9 + \cdots)^7$.

(b) $(x + x^2 + x^3 + x^4)^5$.

(c) $x^2(1 - x)^{12}$.

Solutions

(a) $(1 + x^3 + x^6 + x^9 + \dots)^7$

We use the identity

$$1 + x^3 + x^6 + \dots = \frac{1}{1 - x^3},$$

so

$$(1 + x^3 + x^6 + \dots)^7 = (1 - x^3)^{-7} = \sum_{m=0}^{\infty} \binom{7+m-1}{m} (x^3)^m = \sum_{m=0}^{\infty} \binom{6+m}{m} x^{3m}.$$

To pick out the coefficient of x^{12} , set $3m = 12$, so $m = 4$. Hence

$$[x^{12}] (1 + x^3 + x^6 + \dots)^7 = \binom{6+4}{4} = \binom{10}{4} = 210.$$

(b) $(x + x^2 + x^3 + x^4)^5$

Factor out x :

$$(x + x^2 + x^3 + x^4)^5 = x^5 (1 + x + x^2 + x^3)^5 = x^5 \left(\frac{1-x^4}{1-x}\right)^5 = x^5 (1 - x^4)^5 (1 - x)^{-5}.$$

Expand $(1 - x^4)^5 = \sum_{j=0}^5 (-1)^j \binom{5}{j} x^{4j}$ and $(1 - x)^{-5} = \sum_{k=0}^{\infty} \binom{4+k}{4} x^k$. The coefficient of x^{12} in the product is the coefficient of x^7 in $(1 - x^4)^5 (1 - x)^{-5}$:

$$[x^{12}] = [x^7] \sum_{j=0}^5 (-1)^j \binom{5}{j} x^{4j} \sum_{k=0}^{\infty} \binom{4+k}{4} x^k = \sum_{j=0}^{\lfloor 7/4 \rfloor} (-1)^j \binom{5}{j} \binom{4+(7-4j)}{4}.$$

Only $j = 0, 1$ contribute:

$$\begin{aligned} j = 0 : & \quad \binom{5}{0} \binom{11}{4} = 330, \\ j = 1 : & \quad - \binom{5}{1} \binom{7}{4} = -175. \end{aligned}$$

Therefore

$$[x^{12}] (x + x^2 + x^3 + x^4)^5 = 330 - 175 = 155.$$

(c) $x^2 (1 - x)^{12}$

Write out the binomial expansion:

$$x^2 (1 - x)^{12} = x^2 \sum_{n=0}^{12} \binom{12}{n} (-1)^n x^n = \sum_{n=0}^{12} \binom{12}{n} (-1)^n x^{n+2}.$$

The coefficient of x^{12} comes from $n + 2 = 12$, i.e. $n = 10$:

$$[x^{12}] = \binom{12}{10} (-1)^{10} = \binom{12}{2} = 66.$$

Hence

$$[x^{12}] x^2 (1 - x)^{12} = 66.$$

10 Catalan Numbers

10.1 Introduction and Motivating Problem

How many ways can we correctly place n pairs of parentheses? This is a classic combinatorial question about **valid parentheses combinations**. For example, with $n = 1$ pair, there is only one valid arrangement: $()$. With $n = 2$ pairs, there are two valid arrangements: $()()$ and $(())$. With $n = 3$ pairs, there are five valid arrangements (e.g. $()()()$, $()(())$, $(())()$, $(()())$, $((()))$). In general, the number of distinct well-formed parentheses sequences grows quickly with n .

This problem was our motivation to study a famous sequence of numbers. (It has connection to a similar LeetCode programming problem about generating parentheses.) The numbers counting valid parentheses structures for $n = 1, 2, 3, \dots$ are:

$$1, 2, 5, 14, 42, 132, \dots$$

These are known as the **Catalan numbers**. In these notes, we will explore Catalan numbers, their recursive definition, how they relate to binary tree structures, and a derivation of their formula using generating functions.

10.2 Definition of Catalan Numbers

The Catalan numbers C_n can be defined recursively. For $n = 0$ (zero pairs of parentheses), we define $C_0 = 1$ by convention (there is exactly one valid arrangement of zero pairs: an empty sequence). For $n \geq 1$, the Catalan number C_n satisfies the recursion:

$$C_n = \sum_{k=0}^{n-1} C_k \cdot C_{n-1-k}.$$

In other words, each C_n is obtained by summing over all products $C_k \cdot C_{n-1-k}$ for $0 \leq k \leq n-1$. This recurrence is the heart of what makes Catalan numbers arise in so many combinatorial structures. We will soon see why this recurrence formula makes sense in terms of counting valid parentheses or binary tree configurations.

*(Additional note: There is also a direct formula for C_n which we will derive later. The first few values $C_0 = 1, C_1 = 1, C_2 = 2, C_3 = 5, C_4 = 14$ confirm the sequence given above.)**

Combinatorial reasoning (parenthesis perspective)

Why does the above recurrence hold for valid parentheses? Consider a valid parentheses sequence of n pairs. Focus on the very first “(” character. It must have a matching “)”. Say this matching “)” occurs after forming k pairs inside (between this “(” and its matching “)”) – those k pairs inside must themselves form a valid sequence. The remaining parentheses (after the matching “)”) will form another valid sequence with $n - 1 - k$ pairs. See the schematic below for a sequence split by the first pair:

$$\underbrace{(\quad S_{\text{inside}} \quad)}_{1 + k \text{ pairs}} S_{\text{outside}},$$

where S_{inside} is a valid sequence of k pairs, and S_{outside} is a valid sequence of $n - 1 - k$ pairs. Any valid sequence can be uniquely decomposed in this way. There are C_k possibilities for S_{inside} and C_{n-1-k} possibilities for S_{outside} . Multiplying and summing over all k from 0 to $n-1$ gives the recurrence $C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}$, as stated above.

*(Additional note: The same recurrence will be explained again using binary trees below, which is an equivalent interpretation. The key idea is splitting a structure (parentheses or tree) at a certain point, resulting in two smaller independent structures.)**

10.3 Catalan Numbers and Binary Trees

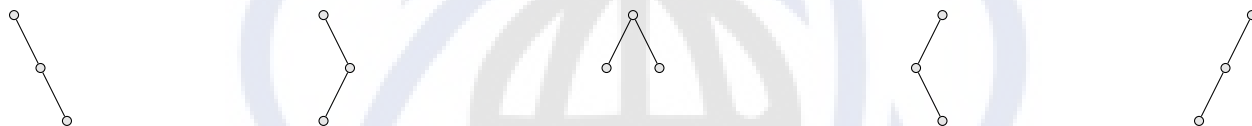
Catalan numbers also count the number of distinct **binary tree** structures with a given number of nodes. To appreciate this connection, we first review what a binary tree is:

A **binary tree** is a hierarchical structure consisting of nodes, where each node may have up to two children: a left child and a right child. A node with no children is called a **leaf**. We often draw binary trees in a planar way with the root at the top, left children branching to the left, and right children to the right. In counting binary trees for Catalan numbers, we consider different shapes of the tree (different arrangements of nodes and child connections) as distinct, but we do **not** label the nodes with any specific values.

For example, with only 1 node, there is exactly one possible binary tree (just the root by itself). With 2 nodes, there are exactly two distinct binary tree shapes:



In the first tree above, the root has a left child but no right child. In the second tree, the root has a right child but no left child. These are the only two possible configurations for 2 nodes. Now, for 3 nodes, it turns out there are 5 distinct binary tree shapes. We can enumerate all five (to visualize them, each diagram below shows the shape, with nodes represented by circles):



(All 5 distinct binary trees with 3 nodes)

For 3 nodes, the five tree shapes can be described as: 1. A right-skewed chain (root $\rightarrow$ right child $\rightarrow$ right grandchild). 2. A tree where the root has only a right child, and that child in turn has a left child. 3. A balanced tree (root with one left child and one right child). 4. A tree where the root has only a left child, and that child has a right child. 5. A left-skewed chain (root $\rightarrow$ left child $\rightarrow$ left grandchild).

If we proceed to 4 nodes, the number of distinct binary trees grows to 14. Detailing all 14 shapes is cumbersome, but we can categorize them by how the root splits the nodes between left and right subtrees: - 5 of those trees have all 3 of the other nodes in the left subtree (and right subtree empty), and another 5 have all 3 in the right subtree (left empty). These 10 are essentially a root with one side empty and the other side being one of the 5 shapes from the 3-node case. - 2 of the trees have the root with 1 node in the left subtree and 2 in the right subtree. - The remaining 2 have 2 nodes in the left subtree and 1 in the right subtree.

Adding these cases: $5 + 5 + 2 + 2 = 14$ total shapes for 4 nodes. (Indeed, 14 is the next Catalan number after 5.)

This pattern is no coincidence. In fact, the recurrence relation for C_n can be understood by considering how a binary tree of n nodes can be formed. Suppose a binary tree has n nodes in total. Pick a number k between 0 and $n - 1$ (inclusive) to be the number of nodes in the left subtree of the root. Then the right subtree will have $n - 1 - k$ nodes (since one node is the root itself). There are C_k possible shapes for the left subtree (by definition of Catalan numbers for k nodes) and C_{n-1-k} possible shapes for the right subtree. These choices are independent, so there are $C_k \cdot C_{n-1-k}$ possible trees with that particular split of k and $n - 1 - k$. Summing over all $k = 0, 1, 2, \dots, n - 1$ gives exactly $C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}$. This is the same recursive formula we encountered earlier, now interpreted in terms of binary trees. Thus, the number of valid parentheses arrangements with n pairs is equal to the number of binary tree structures with n nodes, and both are given by the n th Catalan number.

10.4 Deriving the Formula using Generating Functions

While the recursive definition of Catalan numbers is useful, we can go further and derive a closed-form formula for C_n . A powerful method to solve such recurrences is to use a **generating function**. Define the generating function $C(x)$ for the Catalan sequence as:

$$C(x) = C_0 + C_1x + C_2x^2 + C_3x^3 + \cdots = \sum_{n \geq 0} C_n x^n.$$

Using the recurrence $C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}$, we can derive an equation for $C(x)$. First, note that:

$$C(x) - C_0 = \sum_{n \geq 1} C_n x^n = \sum_{n \geq 1} \left(\sum_{k=0}^{n-1} C_k C_{n-1-k} \right) x^n.$$

Now, change the summation index by letting $m = n - 1$. Then $n \geq 1$ corresponds to $m \geq 0$, and $n = m + 1$. The above becomes:

$$C(x) - 1 = \sum_{m \geq 0} \left(\sum_{k=0}^m C_k C_{m-k} \right) x^{m+1} = x \sum_{m \geq 0} \sum_{k=0}^m C_k C_{m-k} x^m.$$

But the double sum $\sum_{m \geq 0} \sum_{k=0}^m C_k C_{m-k} x^m$ is recognized as the product of two power series. In fact, by the Cauchy convolution formula,

$$\sum_{m \geq 0} \sum_{k=0}^m C_k C_{m-k} x^m = \left(\sum_{k \geq 0} C_k x^k \right) \left(\sum_{j \geq 0} C_j x^j \right) = C(x) C(x) = [C(x)]^2.$$

Therefore, we have the following functional equation for $C(x)$:

$$C(x) - 1 = x [C(x)]^2,$$

or equivalently,

$$C(x) = 1 + x [C(x)]^2.$$

This equation is derived directly from the Catalan recurrence. Now we solve for $C(x)$ as an explicit function of x . The equation $C(x) = 1 + x[C(x)]^2$ can be rearranged into a quadratic equation in $C(x)$:

$$x[C(x)]^2 - C(x) + 1 = 0.$$

Solving this quadratic for $C(x)$, we use the quadratic formula (treating $C(x)$ as the unknown and x as a constant):

$$C(x) = \frac{1 \pm \sqrt{1 - 4x}}{2x}.$$

There are two solutions, but we must choose the one that gives a valid power series expansion. Since $C(0)$ should equal $C_0 = 1$, we take the **negative** branch of the $\pm$ (this ensures $C(x)$ is finite at $x = 0$):

$$C(x) = \frac{1 - \sqrt{1 - 4x}}{2x}.$$

This is the generating function for the Catalan numbers. We can expand this to obtain a formula for C_n . The series expansion of the square root can be derived using the binomial series:

$$\sqrt{1 - 4x} = 1 - 2x - 2x^2 - 4x^3 - 8x^4 - \cdots,$$

but a more straightforward way is to recognize the known power series for Catalan numbers. In fact, the coefficient extraction can be done by comparing with the binomial theorem. The final result (which one can derive by expanding or by known combinatorial identities) is:

$$C_n = \frac{1}{n+1} \binom{2n}{n},$$

for $n \geq 0$. This elegant formula gives the n th Catalan number directly. For example, for $n = 4$ it gives $C_4 = \frac{1}{5} \binom{8}{4} = \frac{1}{5} \cdot 70 = 14$, consistent with our earlier count of binary trees or parentheses combinations.

(Additional note: The closed-form formula above was not explicitly given in the lecture, but it is a well-known result for Catalan numbers. It can be proven by induction or other methods as well. Catalan numbers appear in numerous other combinatorial problems, such as counting paths in a grid, polygon triangulations, full binary trees with $n + 1$ leaves, and many more.)

11 Encoding and Decoding Functions

11.1 Encoding and Decoding Functions

Let p be a prime number. We work over the finite field $\mathbb{Z}_p$ (the integers mod p). For any positive integer n , the set $\mathbb{Z}_p^n$ denotes the set of all n -tuples of elements from $\mathbb{Z}_p$. Note that since $\mathbb{Z}_p$ is a field, the spaces $\mathbb{Z}_p^n$ and $\mathbb{Z}_p^k$ can be viewed as vector spaces (of dimension n or k respectively) over $\mathbb{Z}_p$.

Encoding function: A map $\xi : \mathbb{Z}_p^k \rightarrow \mathbb{Z}_p^n$ is called an *encoding function* (or *coding function*) if it is one-to-one (distinct messages have distinct codewords). In other words, ξ takes each possible message (an element of $\mathbb{Z}_p^k$) and encodes it as a longer string (an element of $\mathbb{Z}_p^n$). The image of ξ , denoted

$$\mathcal{C} = \xi(\mathbb{Z}_p^k) \subseteq \mathbb{Z}_p^n,$$

is the set of all possible encoded messages and is called the *code*. The elements of $\mathcal{C}$ are called *codewords*. Working over the field $\mathbb{Z}_p$ is crucial here, because it ensures we can use the tools of linear algebra and arithmetic modulo p when designing and analyzing codes.

Decoding function: Given an encoding ξ , a map $\eta : \mathbb{Z}_p^n \rightarrow \mathbb{Z}_p^k$ is called a *decoding function* for ξ if $\eta(\xi(u)) = u$ for every $u \in \mathbb{Z}_p^k$. In other words, η is a (left) inverse of ξ on the set of codewords, so that decoding an encoded message returns the original message (assuming no errors in transmission).

Once a message $u \in \mathbb{Z}_p^k$ is encoded as a codeword $c = \xi(u) \in \mathbb{Z}_p^n$, it is sent through a channel (transmission). During transmission, some components of c might be altered due to noise or other errors.

11.2 Error Vectors and Transmission Errors

When a codeword is transmitted (over a noisy channel, or stored and later retrieved), some of its components may change due to errors. We can model the effect of errors by an *error vector*. An *error vector* $\mathbf{e} \in \mathbb{Z}_p^n$ is a vector that, when added to the original codeword, yields the *received word*. If $\mathbf{c} \in \mathcal{C}$ is the sent codeword and $\mathbf{e}$ is the error vector, then the received word is

$$\mathbf{r} = \mathbf{c} + \mathbf{e},$$

where addition is component-wise in $\mathbb{Z}_p$. (In the case $p = 2$, this addition is XOR of bits.) The nonzero entries of $\mathbf{e}$ indicate positions where an error occurred (and the value indicates what was added at that position, e.g. a flip from 0 to 1 in binary is represented by adding 1 mod 2).

The number of errors that occur is the number of nonzero entries in $\mathbf{e}$. This is called the *weight* of $\mathbf{e}$, often denoted $w(\mathbf{e})$. If $w(\mathbf{e}) = t$, we say t errors occurred. The coding and decoding process can be summarized as:

$$\mathbf{u} \xrightarrow{\xi} \mathbf{c} \xrightarrow{\text{error } \mathbf{e}} \mathbf{r} = \mathbf{c} + \mathbf{e} \xrightarrow{\eta} \hat{\mathbf{u}},$$

where $\hat{\mathbf{u}} = \eta(\mathbf{r})$ is the decoder's estimate of the original message. We desire $\hat{\mathbf{u}} = \mathbf{u}$ even if $\mathbf{e}$ is nonzero (up to a certain weight). The design of $\mathcal{C}$ and η determines how many errors can be reliably detected or corrected.

11.3 Examples of Encoding Functions (Codes)

We now present a few example encoding functions and their corresponding codes:

Example 1: Repetition Code

One simple encoding is the *repetition code*. Here the message space is $\mathbb{Z}_p$ (messages of length 1). The encoding function $\xi_1 : \mathbb{Z}_p \rightarrow \mathbb{Z}_p^n$ for some chosen length n is defined by

$$\xi_1(a) = (\underbrace{a, a, \dots, a}_{n \text{ times}}),$$

i.e. the single-symbol message a is repeated n times to form the codeword. The code $\mathcal{C}_1 = \{(a, a, \dots, a) : a \in \mathbb{Z}_p\}$ consists of all n -tuples with identical entries. This encoding is one-to-one (different a give different constant tuples).

For example, over $\mathbb{Z}_2$ (the binary case), if $a = 1$ and $n = 5$, the codeword would be $(1, 1, 1, 1, 1)$. If a transmission error flips one of these bits (say we receive $(1, 1, 1, 0, 1)$), the decoder can notice that this is not a valid codeword in $\mathcal{C}_1$ (since not all bits are the same), hence an error is detected. In fact, the repetition code is able to detect up to $n - 1$ errors (any change in at most $n - 1$ positions will yield a tuple that is not constant and thus not in the code). This code even has error-correcting capability: intuitively, the decoder could guess that the majority value is the intended bit (for example, from $(1, 1, 1, 0, 1)$ a reasonable decoded value for a would be 1, since most bits are 1). However, our focus here is on error detection criteria, which we formalize later.

Example 2: Parity Check Code

Another useful encoding adds a *parity check* to the message. For this example, let's work in $\mathbb{Z}_p$ (and especially consider $p = 2$ for binary parity). Let the message space be $\mathbb{Z}_p^{k-1}$ (messages of length $k - 1$). Define an encoding function $\xi_2 : \mathbb{Z}_p^{k-1} \rightarrow \mathbb{Z}_p^k$ by

$$\xi_2(a_1, a_2, \dots, a_{k-1}) = (a_1, a_2, \dots, a_{k-1}, a_k),$$

where the last component a_k is chosen such that the total sum of the k components is 0 in $\mathbb{Z}_p$. In other words,

$$a_k = -(a_1 + a_2 + \dots + a_{k-1}) \pmod{p},$$

which ensures $a_1 + a_2 + \dots + a_{k-1} + a_k \equiv 0 \pmod{p}$. This extra symbol a_k is a redundancy (the *parity symbol*) chosen to enforce a constraint on every codeword.

The resulting code is

$$\mathcal{C}_2 = \{(a_1, \dots, a_{k-1}, a_k) \in \mathbb{Z}_p^k : a_1 + \dots + a_{k-1} + a_k = 0\}.$$

In the special case $p = 2$ (binary), this encoding is the standard single-parity-bit code: the last bit a_k is 0 or 1 chosen to make the total number of 1's in the codeword even. For example, if the message is $(1, 0, 1, 1)$ in $\mathbb{Z}_2^4$ (so $k - 1 = 4$), then the parity bit a_5 is chosen so that $1 + 0 + 1 + 1 + a_5 \equiv 0 \pmod{2}$. Here $1 + 0 + 1 + 1 = 3 \equiv 1 \pmod{2}$, so we must take $a_5 = 1$ to make the sum even. The encoded 5-bit codeword is $(1, 0, 1, 1, 1)$, which has an even number of 1's (four 1's in this case).

This parity check code allows detection of any single-error in transmission: if one bit of a codeword is flipped, the parity (sum mod p) will no longer be zero, so the received word will violate the code constraint and thus be recognized as invalid. For instance, if $(1, 0, 1, 1, 1)$ is sent and one bit flips, the sum of bits will be odd, alerting us to the error. In fact, more generally, if an odd number of bits are in error, the parity check will detect it (sum becomes nonzero mod 2). However, if an even number of bits are corrupted in the binary case, the parity check could fail to detect it (since an even number of flips can restore the sum to even).

Example 3: PESEL Check Digit (revised)

The Polish national identification number (PESEL) is an 11-digit string

$$YYMMDDPPPPK,$$

where

- $YYMMDD$ is the date of birth.
- $PPPP$ is a serial number; its last digit is *even* for females and *odd* for males.
- K is the check digit making

$$\sum_{i=1}^{10} w_i p_i + K \equiv 0 \pmod{10}, \quad (w_i)_{i=1}^{10} = 1, 3, 7, 9, 1, 3, 7, 9, 1, 3.$$

Example. A woman born on 15 April 1997 with serial 1478 has the preliminary digits 9704151478. The weighted sum is $1 \cdot 9 + 3 \cdot 7 + \dots + 3 \cdot 8 = 156 \equiv 6 \pmod{10}$; hence $K = (10 - 6) \bmod 10 = 4$. The complete PESEL is 97041514784.

Error detection. This checksum catches *every* single-digit error and the overwhelming majority of adjacent transposition errors.

For the first ten digits $(p_1, \dots, p_{10}) \in \mathbb{Z}_{10}^{10}$ define

$$\xi_3 : \mathbb{Z}_{10}^{10} \longrightarrow \mathbb{Z}_{10}^{11}, \quad \xi_3(p_1, \dots, p_{10}) = (p_1, \dots, p_{10}, p_{11}),$$

where

$$p_{11} = (10 - \sum_{i=1}^{10} w_i p_i) \bmod 10, \quad (w_i)_{i=1}^{10} = 1, 3, 7, 9, 1, 3, 7, 9, 1, 3.$$

Because ξ_3 is injective, its image $\mathcal{C}_3 = \xi_3(\mathbb{Z}_{10}^{10}) \subset \mathbb{Z}_{10}^{11}$ is a legitimate code—the PESEL code—whose distance properties are governed by the checksum just illustrated.

11.4 Hamming Distance

We need a way to quantify how “different” two vectors (e.g. the transmitted and received words) are. The appropriate notion is the *Hamming distance*. For vectors $\mathbf{x}, \mathbf{y} \in \mathbb{Z}_p^n$, the **Hamming distance** $d_H(\mathbf{x}, \mathbf{y})$ is defined as the number of coordinates in which $\mathbf{x}$ and $\mathbf{y}$ differ. Equivalently,

$$d_H(\mathbf{x}, \mathbf{y}) = |\{i : 1 \leq i \leq n, x_i \neq y_i\}|.$$

For example, in binary, $d_H(01011, 01101) = 2$ (they differ in two bit positions).

It is easy to check that Hamming distance satisfies the properties of a distance metric on $\mathbb{Z}_p^n$. In particular, for any $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathbb{Z}_p^n$:

1. $d_H(\mathbf{x}, \mathbf{y}) \geq 0$, and $d_H(\mathbf{x}, \mathbf{y}) = 0$ if and only if $\mathbf{x} = \mathbf{y}$.
2. $d_H(\mathbf{x}, \mathbf{y}) = d_H(\mathbf{y}, \mathbf{x})$ (symmetry).
3. $d_H(\mathbf{x}, \mathbf{z}) \leq d_H(\mathbf{x}, \mathbf{y}) + d_H(\mathbf{y}, \mathbf{z})$ (triangle inequality).

Property (3) means that for any two vectors $\mathbf{x}$ and $\mathbf{z}$, any differences between $\mathbf{x}$ and $\mathbf{z}$ must show up when comparing $\mathbf{x}$ to some intermediate $\mathbf{y}$ or $\mathbf{y}$ to $\mathbf{z}$. In other words, any coordinate where $\mathbf{x}$ and $\mathbf{z}$ differ is either a coordinate where $\mathbf{x}$ differs from $\mathbf{y}$ or $\mathbf{y}$ differs from $\mathbf{z}$ (or both). This ensures no “shortcut” can make d_H violate the triangle inequality. Thus, d_H is a valid distance function.

The Hamming distance gives us a natural way to describe the effect of errors: if a codeword $\mathbf{c}$ is transmitted and a word $\mathbf{r}$ is received, then $d_H(\mathbf{c}, \mathbf{r}) = w(\mathbf{e})$, the weight of the error vector (the number of errors that occurred). For a given code $C \subseteq \mathbb{Z}_p^n$, we often speak of the distance *between codewords* measured by d_H . The **minimum distance** of code C , denoted $d(C)$ or d_C , is the smallest Hamming distance between any two distinct codewords in C :

$$d_C := \min\{d_H(\mathbf{c}_1, \mathbf{c}_2) : \mathbf{c}_1, \mathbf{c}_2 \in C, \mathbf{c}_1 \neq \mathbf{c}_2\}.$$

11.5 Metric Spaces and Other Distance Metrics

The pair $(\mathbb{Z}_p^n, d_H)$ is an example of a *metric space*. In general, a **metric space** is a tuple (X, d) , where X is a set and

$$d : X \times X \longrightarrow \mathbb{R}$$

is a function (called a *metric*) satisfying the three properties listed above (non-negativity and identity of indiscernibles, symmetry, and the triangle inequality). The value $d(x, y)$ represents the “distance” between points x and y in the space.

There are many examples of metrics aside from the Hamming distance. A few common metrics are:

- **Euclidean distance.** On $\mathbb{R}^n$,

$$d_E(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2},$$

the usual “straight-line” distance from geometry derived from the Pythagorean theorem.

- **Maximum (Chebyshev) distance.** Also on $\mathbb{R}^n$ or $\mathbb{Z}^n$,

$$d_\infty(\mathbf{x}, \mathbf{y}) = \max_{1 \leq i \leq n} |x_i - y_i|.$$

Here distance is determined by the single largest coordinate-difference. Intuitively, the distance is measured in terms of the largest single-coordinate difference. For example, in the plane $\mathbb{R}^2$, the distance between (x_1, y_1) and (x_2, y_2) is $\max(|x_2 - x_1|, |y_2 - y_1|)$, which in effect produces a square-shaped notion of distance (all points with d_∞ distance $\leq r$ from a center form a square of side length $2r$).

Many other metrics exist (Manhattan distance, discrete metric, etc.), but the Hamming distance is particularly useful for analyzing codes. We will now focus on metric properties in the context of coding theory.

11.6 Balls and Error Correction

Given a metric space (X, d) , a **ball** (or *distance ball*) of radius r centered at a point $x \in X$ is the set

$$B_r(x) = \{y \in X : d(x, y) \leq r\}.$$

It contains all points that are at distance at most r from x . In the Hamming metric space $(\mathbb{Z}_p^n, d_H)$, the ball of radius r around a vector $\mathbf{x}$ consists of all vectors that differ from $\mathbf{x}$ in at most r coordinate positions. For example, in $(\mathbb{Z}_2)^3$ (3-bit binary space),

$$B_1(000) = \{000, 001, 010, 100\},$$

since these are exactly the 3-bit vectors with at most 1 bit different from 000. The size of $B_r(x)$ in a Hamming space depends on r and n ; for instance, in $(\mathbb{Z}_2)^3$, a radius-1 ball has $1 + \binom{3}{1} = 4$ elements (including the center).

In coding theory, metric balls are a useful way to visualize and implement error correction. Suppose we have a code $C \subseteq \mathbb{Z}_p^n$. To *correct up to t errors*, the decoding process can be viewed as follows: given a received word $\mathbf{r}$, find the codeword $\mathbf{c} \in C$ that is closest to $\mathbf{r}$ in Hamming distance (i.e. with minimal $d_H(\mathbf{c}, \mathbf{r})$). If the number of errors is at most t , and if certain distance conditions hold, this closest codeword will be the one that was originally sent. In essence, we imagine around each codeword $\mathbf{c}$ a *ball of radius t* . If no two codewords' radius- t balls overlap, then any received word that lies within distance t of a codeword $\mathbf{c}$ could only have come from $\mathbf{c}$ (because if it were also within t of a different codeword $\mathbf{c}'$, then the balls would intersect at $\mathbf{r}$). In that case, we can unambiguously decode $\mathbf{r}$ to $\mathbf{c}$. On the other hand, if the balls of radius t around codewords overlap or touch, a received word could be close to two different codewords, causing ambiguity in decoding.

Metric balls also give a criterion for *detecting* errors (even if we cannot correct them). A code can *detect* up to t errors if whenever up to t errors occur, the result is *not* a valid codeword (unless no error occurred). In terms of distance, this means that if you take any codeword $\mathbf{c}$ and look at all vectors within distance t of $\mathbf{c}$ (excluding $\mathbf{c}$ itself), none of those vectors are codewords in C . Equivalently, the balls of radius t around each codeword $\mathbf{c} \in C$, *not counting the center*, do not contain any other codeword. (They may overlap with balls around other codewords, but only at non-codeword points, which corresponds to errors causing an invalid sequence that the receiver can recognize as erroneous.)

11.7 A Geometric Example: $(\mathbb{Z}_2)^3$ as a Hamming Space

To visualize these concepts, consider the 3-dimensional binary vector space $(\mathbb{Z}_2)^3$ with the Hamming metric. This space consists of 8 vectors (from 000 to 111). We can geometrically represent $(\mathbb{Z}_2)^3$ as the vertices of a cube, where edges connect vectors that differ in exactly one coordinate (Hamming distance 1 apart). In the figure below, each vertex is labeled by the corresponding 3-bit vector, and each edge represents a single-bit flip. This graph is often called the 3-dimensional *Hamming cube*.

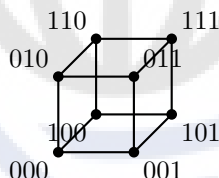


Figure 4: Each vertex corresponds to a 3-bit binary vector. Edges connect vertices that differ in one coordinate (Hamming distance 1). For example, the neighbors of 000 (connected directly by an edge) are 001, 010, and 100, which illustrates all vectors at Hamming distance 1 from 000.

In this cube, one can see how clusters of vectors can be grouped around a given vertex. For instance, as noted, the set $\{000, 001, 010, 100\}$ are all within distance 1 of 000 (a Hamming ball of radius 1 around 000). If we take a simple code such as $C = \{000, 111\}$ (which is the $((3, 1))$ repetition code in binary with $n = 3$, $k = 1$), those codewords correspond to the vertices 000 and 111, which are opposite corners of the cube. The minimum distance of this code is $d_C = d_H(000, 111) = 3$. The balls of radius 1 around 000 and around 111 (the sets of neighbors each has) do not overlap; in fact, they are separated by at least one bit flip difference. Thus, if at most 1 error occurs during transmission, the received word will remain in the unique “sphere” of radius 1 around whichever codeword was sent, and decoding can correctly deduce the original codeword. If 2 errors occur, the received word will be distance 2 from the original codeword — in this case (for $C = \{000, 111\}$) a double error could yield a vector like 110, which is not a codeword but is now only distance 1 away from the *other* codeword 111. The decoder might misidentify it if it only considers radius 1 spheres, but it will at least notice an error occurred since 110 is not itself a codeword.

11.8 Error Detection and Correction Bounds

Before looking at concrete codes, it is helpful to understand in general how the minimum distance of a code determines its ability to detect and correct errors.

1. Minimum distance. Let $C \subseteq \mathbb{Z}_p^n$ be a code with minimum Hamming distance

$$d_{\min} = \min_{\substack{\mathbf{c}_1, \mathbf{c}_2 \in C \\ \mathbf{c}_1 \neq \mathbf{c}_2}} d_H(\mathbf{c}_1, \mathbf{c}_2).$$

By definition, any two distinct codewords differ in at least $d_{\min}$ positions.

2. Error detection. If up to t errors occur, then the received word $\mathbf{r} = \mathbf{c} + \mathbf{e}$ satisfies

$$d_H(\mathbf{c}, \mathbf{r}) = w(\mathbf{e}) \leq t.$$

To *detect* every such error, $\mathbf{r}$ must never coincide with a different codeword. In other words, no two codewords may be within distance t of one another. Since the closest pair of distinct codewords are $d_{\min}$ apart, we need

$$t < d_{\min}.$$

Hence a code with minimum distance $d_{\min}$ can detect all error patterns of weight up to

$$t_{\det} = d_{\min} - 1.$$

3. Error correction. To *correct* up to t errors, a decoder typically chooses the codeword nearest to the received word $\mathbf{r}$. Geometrically, this means the Hamming balls of radius t around each codeword must be disjoint: if two balls overlap, a received word in the overlap could have come from either center, making correct decoding impossible.

Two balls of radius t around codewords $\mathbf{c}_1$ and $\mathbf{c}_2$ remain disjoint exactly when

$$2t < d_H(\mathbf{c}_1, \mathbf{c}_2) \quad \text{for all } \mathbf{c}_1 \neq \mathbf{c}_2.$$

Since the minimum distance between any two distinct codewords is $d_{\min}$, the strongest requirement is

$$2t < d_{\min}.$$

Therefore, the maximum number of errors that can be *corrected* is

$$t_{\text{corr}} = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor.$$

Summary. If a code C has minimum Hamming distance $d_{\min}$, then

$$t_{\det} = d_{\min} - 1, \quad t_{\text{corr}} = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor.$$

These simple formulas allow us, once we know $d_{\min}$, to immediately read off how many errors the code can detect and how many it can correct.

Example 1

Consider the 5-bit binary code

$$C = \{00000, 01011, 10101, 11110\} \subset (\mathbb{Z}_2)^5.$$

Checking pairwise Hamming distances gives

$$d_H(00000, 01011) = 3, \quad d_H(00000, 10101) = 3, \quad d_H(01011, 10101) = 4, \dots$$

so the minimum distance is

$$d_{\min} = 3.$$

Hence

$$t_{\det} = 3 - 1 = 2, \quad t_{\text{corr}} = \lfloor (3 - 1)/2 \rfloor = 1.$$

Conclusion: this code can detect up to 2 errors and correct up to 1 error.

Example 2

Now take the 6-bit code

$$C = \{000000, 010101, 101010, 111111\} \subset (\mathbb{Z}_2)^6.$$

One finds quickly that every pair of distinct codewords differs in exactly 3 or 6 positions, so

$$d_{\min} = 3.$$

Thus again

$$t_{\det} = 2, \quad t_{\text{corr}} = 1.$$

Example 3: Repetition Code of Length 7

Finally, the binary repetition code of length 7 is

$$C = \{ \underbrace{0000000}_{7 \text{ times}}, \underbrace{1111111}_{7 \text{ times}} \}.$$

Clearly any two distinct codewords differ in all 7 positions, so

$$d_{\min} = 7.$$

Therefore

$$t_{\det} = 7 - 1 = 6, \quad t_{\text{corr}} = \lfloor (7 - 1)/2 \rfloor = 3.$$

Conclusion: the 7-fold repetition code can detect up to 6 errors and correct up to 3 errors.

11.9 Detection, Correction, and Perfect Codes

Theorem

A code C is able to detect t errors or fewer iff the minimum distance satisfies $d_C \geq t + 1$,

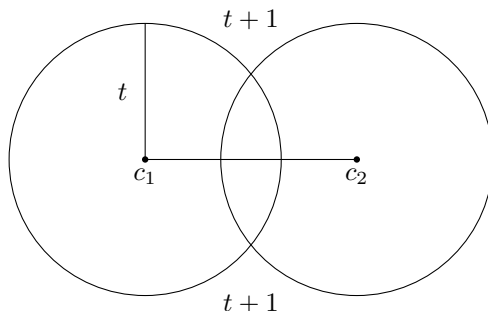
$$d_C = \min\{d_H(c_1, c_2) : c_1, c_2 \in C, c_1 \neq c_2\}.$$

Proof

($\Rightarrow$) Let $c_1 \in C$ be the original codeword sent, and let f be the received word. Assume $d_C \geq t + 1$. If at most t errors occurred, then $d_H(c_1, f) \leq t$. For any *other* codeword $c_2 \in C$ with $c_2 \neq c_1$ we have

$$d_H(c_2, f) \geq d_H(c_1, c_2) - d_H(c_1, f) \geq (t + 1) - t = 1.$$

Hence $f \notin C$, so the decoder recognises that an error happened; the error is *detectable*.



($\Leftarrow$) Suppose the code detects every pattern of at most t errors. For any $c \in C$ and word f with $d_H(c, f) \leq t$, we must have $f \notin C$ unless $f = c$. Thus if $c, c' \in C$ are distinct, c' cannot lie within distance t of c ; otherwise c' would be a second codeword in the ball of radius t around c . Therefore $d_H(c, c') > t$ for all $c \neq c'$, hence (integer distance) $d_H(c, c') \geq t + 1$. Taking the minimum over all pairs gives $d_C \geq t + 1$. $\square$

Definition

A code C *corrects t errors or fewer* if, whenever $d_H(c, f) \leq t$ for some $c \in C$, that c is the *unique* codeword within distance $\leq t$ of f .

Theorem

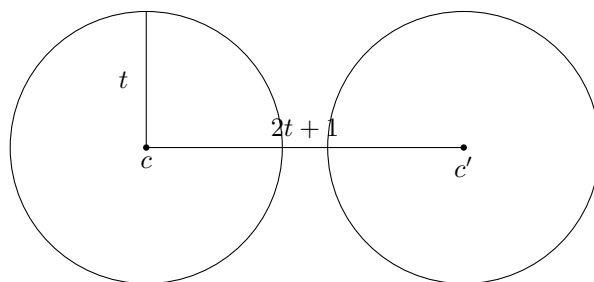
A code C corrects t or fewer errors *iff*

$$d_C \geq 2t + 1.$$

Corollary

C corrects t errors or fewer *iff*

$$B(c, t) \cap B(c', t) = \emptyset \quad \text{for all } c \neq c' \in C.$$



Example

Let

$$\varphi : \mathbb{Z}_p^{n-1} \longrightarrow \mathbb{Z}_p^n, \quad \varphi(a_1, \dots, a_{n-1}) = (a_1, \dots, a_{n-1}, -\sum_{i=1}^{n-1} a_i).$$

This is the length- n *single-parity-check code*. Its minimum distance is $d_C = 2$.

- **Detection:** $d \geq t + 1$ ($2 \geq t + 1 \Rightarrow t \leq 1$). So the code *detects exactly one error*.
- **Correction:** $d \geq 2t + 1$ ($2 \geq 2t + 1$ impossible for $t \geq 1$). Hence it *cannot correct a single error* (only $t = 0$).

Example (repetition code)

Define

$$\xi : \mathbb{Z}_p \longrightarrow \mathbb{Z}_p^n, \quad \xi(a) = (\underbrace{a, \dots, a}_{n \text{ times}}).$$

This n -fold repetition code has

$$d_C = n, \quad \text{detection capacity } n - 1, \quad \text{correction capacity } \left\lfloor \frac{n-1}{2} \right\rfloor.$$

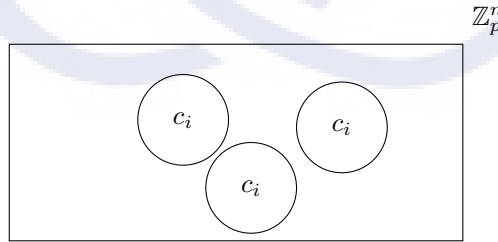
Theorem (Hamming bound)

If a code $C \subseteq \mathbb{Z}_p^n$ with M elements corrects t errors, then

$$M(1 + (p-1)\binom{n}{1} + (p-1)\binom{n}{2} + \dots + (p-1)\binom{n}{t}) \leq p^n.$$

Proof (sketch)

Draw a radius- t Hamming ball around each codeword. Since the balls are disjoint, their union fits inside $\mathbb{Z}_p^n$, giving the inequality.



Definition (perfect code)

A code correcting t errors is *perfect* when equality holds, i.e.

$$M(1 + (p-1)\binom{n}{1} + \dots + (p-1)\binom{n}{t}) = p^n.$$

11.10 Linear Codes

Definition (Linear codes)

Let $\zeta : \mathbb{Z}_p^k \rightarrow \mathbb{Z}_p^n$ be a coding function that maps a message vector of length k to a codeword of length n (where $k < n$). The function ζ is called a linear encoder, if it is a linear mapping between the vector spaces $\mathbb{Z}_p^k$ and $\mathbb{Z}_p^n$. That is, for all vectors $\mathbf{u}, \mathbf{v} \in \mathbb{Z}_p^k$ and all scalars $\alpha, \beta \in \mathbb{Z}_p$, the function satisfies:

$$\zeta(\alpha\mathbf{u} + \beta\mathbf{v}) = \alpha\zeta(\mathbf{u}) + \beta\zeta(\mathbf{v})$$

The image of $\mathbb{Z}_p^k$ under ζ , denoted as $C = \text{Im}(\zeta)$, is a subspace of $\mathbb{Z}_p^n$.

Hence: A non-empty subset $C \subseteq \mathbb{Z}_p^n$ is a linear code if and only if it is closed under linear combinations. That is, for all codewords $\mathbf{c}_1, \mathbf{c}_2 \in C$ and all scalars $\alpha, \beta \in \mathbb{Z}_p$:

$$\alpha\mathbf{c}_1 + \beta\mathbf{c}_2 \in C$$

Example 1

Consider the repetition code

$$C = \left\{ \underbrace{(a, a, \dots, a)}_{n \text{ times}} \mid a \in \mathbb{Z}_p \right\} \subseteq \mathbb{Z}_p^n$$

It is easy to show that C is a linear code.

Proof:

To prove that C is a linear code, we must demonstrate that it is a subspace of $\mathbb{Z}_p^n$. We will use the combined subspace criterion. Let $\mathbf{c}_1, \mathbf{c}_2$ be any two codewords from C , and let α, β be any two scalars from the finite field $\mathbb{Z}_p$. By definition of the set C , since $\mathbf{c}_1 \in C$ and $\mathbf{c}_2 \in C$, there must exist two scalars $a_1, a_2 \in \mathbb{Z}_p$ such that:

$$\begin{aligned} \mathbf{c}_1 &= (a_1, a_1, \dots, a_1) \\ \mathbf{c}_2 &= (a_2, a_2, \dots, a_2) \end{aligned}$$

Now, let us construct the linear combination $\alpha\mathbf{c}_1 + \beta\mathbf{c}_2$:

$$\alpha\mathbf{c}_1 + \beta\mathbf{c}_2 = \alpha(a_1, a_1, \dots, a_1) + \beta(a_2, a_2, \dots, a_2)$$

Using component-wise vector addition and scalar multiplication in $\mathbb{Z}_p^n$, we can combine these terms:

$$\begin{aligned} \alpha\mathbf{c}_1 + \beta\mathbf{c}_2 &= (\alpha a_1, \alpha a_1, \dots, \alpha a_1) + (\beta a_2, \beta a_2, \dots, \beta a_2) \\ \alpha\mathbf{c}_1 + \beta\mathbf{c}_2 &= (\alpha a_1 + \beta a_2, \alpha a_1 + \beta a_2, \dots, \alpha a_1 + \beta a_2) \end{aligned}$$

Let us define a new scalar $a_3 = \alpha a_1 + \beta a_2$. Since $\mathbb{Z}_p$ is a field, it is closed under addition and multiplication, meaning that $a_3 \in \mathbb{Z}_p$. Substituting a_3 back into our vector, we get:

$$\alpha\mathbf{c}_1 + \beta\mathbf{c}_2 = \underbrace{(a_3, a_3, \dots, a_3)}_{n \text{ times}}$$

Since this resulting vector consists of the same scalar a_3 repeated n times, where $a_3 \in \mathbb{Z}_p$, it perfectly matches the definition of the set C . Therefore:

$$\alpha\mathbf{c}_1 + \beta\mathbf{c}_2 \in C$$

Because the linear combination of any two codewords remains within C , the set C is a valid subspace of $\mathbb{Z}_p^n$. Hence, C is a linear code. $\square$

Example 2

Formally, the general parity-check code C is defined as the following set:

$$C = \left\{ \left(a_1, a_2, \dots, a_{n-1}, -\sum_{i=1}^{n-1} a_i \right) \mid a_i \in \mathbb{Z}_p \right\} \subseteq \mathbb{Z}_p^n$$

Proof:

Let $\mathbf{c} = (c_1, c_2, \dots, c_n)$ be an arbitrary vector in $\mathbb{Z}_p^n$. By definition, $\mathbf{c}$ belongs to C if and only if its components satisfy the relation:

$$c_n = -\sum_{i=1}^{n-1} c_i$$

By moving the summation to the left-hand side of the equation, we can rewrite this structural constraint as a single homogeneous linear equation:

$$c_1 + c_2 + \dots + c_{n-1} + c_n = 0$$

Thus, the code C can be explicitly defined as the set of all vectors that satisfy this homogeneous equation:

$$C = \left\{ \mathbf{c} \in \mathbb{Z}_p^n \mid \sum_{i=1}^n c_i = 0 \right\}$$

From fundamental linear algebra, the set of all solutions to any system of homogeneous linear equations over a field always forms a vector subspace of the surrounding vector space. $\square$

Remark

Let

$\zeta : \mathbb{Z}_p^k \rightarrow \mathbb{Z}_p^n$ be a linear encoding function defining a linear code C . Then, there exists a $k \times n$ matrix G with entries in $\mathbb{Z}_p$ such that for every message $\mathbf{u} \in \mathbb{Z}_p^k$, the encoding function can be presented as:

$$\zeta(\mathbf{u}) = \mathbf{u}G$$

The matrix G is called the **generator matrix** of the linear code C .

Remark

Let C be a linear code and let G be its generating matrix. Then, every codeword $\mathbf{c} \in C$ can be expressed as a linear combination of the rows of G .

$$\mathbf{c} \in C \iff \mathbf{c} = \mathbf{u}G \quad \text{for } \mathbf{u} \in \mathbb{Z}_p^k$$

Rephrased:

$$C = \{ \mathbf{u}G \mid \mathbf{u} \in \mathbb{Z}_p^k \}$$

Example 3

$$C = \{(0, 0, 0, 0), (0, 1, 0, 1), (1, 0, 1, 1), (1, 1, 1, 0)\} \subseteq \mathbb{Z}_2^4$$

C is a linear code, since it is closed under linear combinations. Encoding function:

$$\zeta : \mathbb{Z}_2^2 \rightarrow \mathbb{Z}_2^4$$

Generating matrix:

$$G = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

$$(0, 0)G = (0, 0, 0, 0) \in C$$

$$(0, 1)G = (1, 0, 1, 1) \in C$$

$$(1, 0)G = (0, 1, 0, 1) \in C$$

$$(1, 1)G = (1, 1, 1, 0) \in C$$

Notice that G is not unique. For instance, we could also use the following generator matrix:

$$G' = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

or

$$G'' = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

The rows of the generator matrix G must be linearly independent.

Definition (standard form)

We say that a generating matrix G is in **standard form** if is of the form:

$$G = [I_k | P]$$

where I_k is the $k \times k$ identity matrix and P is a $k \times (n - k)$ matrix.

Remark

If G is in standard form, then:

$$\underbrace{(u1, u2, \dots, uk)}_u G = (u1, u2, \dots, uk, \dots)$$

Example

$$C = \left\{ \left(a_1, a_2, \dots, a_{n-1}, -\sum_{i=1}^{n-1} a_i \right) \mid a_i \in \mathbb{Z}_p \right\}$$

A generator matrix for this code in standard form is:

$$G = \left[\begin{array}{cccccc|c} 1 & 0 & 0 & \dots & 0 & -1 \\ 0 & 1 & 0 & \dots & 0 & -1 \\ 0 & 0 & 1 & \dots & 0 & -1 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & 1 & -1 \end{array} \right]$$

$\underbrace{\hspace{10em}}_{I_{n-1}}$

$$(a_1, a_2, \dots, a_{n-1})G = (a_1, a_2, \dots, a_{n-1}, -\sum_{i=1}^{n-1} a_i)$$

Notice that if we define $H = (1, 1, \dots, 1)$, then:

$$\mathbf{c}H^T = 0 \iff \mathbf{c} \in C$$

Definition (parity-check matrix)

Let C be a linear code in $\mathbb{Z}_p^n$, with generating matrix G of size $k \times n$. A matrix H of size $(n - k) \times n$ over $\mathbb{Z}_p$ is called a **parity-check matrix** for C , whenever:

$$\mathbf{c} \in C \iff \mathbf{c}H^T = \mathbf{0}$$

How to obtain H from G?

If G is in standard form, i.e. $G = [I_k | P]$, then a parity-check matrix for C can be constructed as:

$$H = [-P^T | I_{n-k}]$$

H is parity check matrix for C because:

$$\mathbf{c} \in C \iff \mathbf{c} = \mathbf{u}G \text{ for some } \mathbf{u} \in \mathbb{Z}_p^k$$

Now,

$$\mathbf{c}H^T = \mathbf{u}GH^T = \mathbf{u}[I_k | P] [-P^T | I_{n-k}]^T = \mathbf{u}[I_k | P] \begin{bmatrix} -P \\ I_{n-k} \end{bmatrix} = \mathbf{u}(-P + P) = \mathbf{0}$$

If the generating matrix G is not in standard form, the parity-check matrix H can be constructed by solving the homogeneous system of linear equations:

$$G\mathbf{x} = \mathbf{0}$$

Since the solution set $V = \{\mathbf{x} \in \mathbb{Z}_p^n : G\mathbf{x} = \mathbf{0}\}$ forms a subspace of $\mathbb{Z}_p^n$ of dimension $n - k$, it possesses a basis. Let $\{\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_{n-k}\}$ be a basis of this solution space. The parity-check matrix H is then formed by taking these basis vectors as its rows:

$$H = \begin{bmatrix} \mathbf{h}_1 \\ \mathbf{h}_2 \\ \vdots \\ \mathbf{h}_{n-k} \end{bmatrix}$$

By construction, $GH^T = \mathbf{0}$, which ensures that H is a valid parity-check matrix for the code C .

Decoding linear codes

Assume C is a linear code with parity-check matrix H . Moreover assume $\mathbf{c}$ is the codeword sent, but $\mathbf{r} = \mathbf{c} + \mathbf{e}$ was the received word, where $\mathbf{e}$ is the error vector. Then:

$$\mathbf{r}H^T = (\mathbf{c} + \mathbf{e})H^T = \mathbf{c}H^T + \mathbf{e}H^T = \mathbf{0} + \mathbf{e}H^T = \mathbf{e}H^T$$

The product $\mathbf{s} = \mathbf{r}H^T$ is called the syndrome of the received vector $\mathbf{r}$.

Theorem

If C corrects d errors or less and $\mathbf{r} = \mathbf{c} + \mathbf{e}$, where $\mathbf{c} \in C$ is the codeword sent and $\mathbf{e}$ is the error vector, whose weight is at most d , then $\mathbf{e}H^T$ is unique and can be determined by finding all solutions to:

$$\mathbf{r}H^T = \mathbf{x}H^T$$

and choosing the solution $\mathbf{x}$ with the smallest weight.

Example

Let $C = \{(0, 0, 0, 0), (0, 1, 0, 1), (1, 0, 1, 1), (1, 1, 1, 0)\}$ be a linear code with generating matrix:

$$G = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

and parity-check matrix:

$$H = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}$$

Suppose the received word is $\mathbf{r} = (1, 1, 1, 1)$.

$$\mathbf{r}H^T = (1, 1, 1, 1) \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} = [0, 1] \text{ in } \mathbb{Z}_2^2$$

The solutions to $\mathbf{r}H^T = \mathbf{x}H^T$ are:

$$\mathbf{x} = (x_3, 1 + x_3 + x_4, x_3, x_4) \quad \text{for } x_3, x_4 \in \mathbb{Z}_2$$

Vectors from solution set with the smallest weight are:

$$\mathbf{e}_1 = (0, 1, 0, 0)$$

$$\mathbf{e}_2 = (0, 0, 0, 1)$$

Hence, the decoder can choose either $\mathbf{e}_1$ or $\mathbf{e}_2$ as the error vector, and decode $\mathbf{r}$ to $\mathbf{c} = \mathbf{r} - \mathbf{e}_1 = (1, 0, 1, 1)$ or $\mathbf{c} = \mathbf{r} - \mathbf{e}_2 = (1, 1, 1, 0)$, both of which are valid codewords in C .

12 Introduction in Graph Theory

12.1 Formal definition

A graph is a pair $G = (V, E)$ where V is a finite non-empty set (called the set of *vertices*) and E is a set (called the set of *edges*), where $E \subseteq \binom{V}{2}$ (all two-element subsets of V).

Notation:

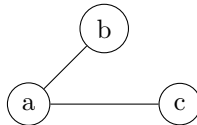
If $G = (V, E)$, then $V =: V(G)$ and $E =: E(G)$.

If $\{u, v\} \in E$, then we write $uv \in E$ ($vu = uv$, as edges are unordered)

If $u, v \in V(G)$ such that $uv \in E(G)$, then we say that u and v are *connected* by an edge in G

Example

Let $G = (V, E)$ where $V = \{a, b, c\}$ and $E = \{\{a, b\}, \{a, c\}\}$. This can be visualized as:



12.2 Basic Concepts and Terminology

Simple Graphs

A graph is called **simple** if it is non-oriented (undirected), has no loops (edges from a vertex to itself), and is not a multigraph (has at most one edge between any pair of vertices). The definition provided in Section 1 refers to simple graphs.

Isomorphism

We say that two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are **isomorphic** if there exists a bijection $\varphi : V_1 \rightarrow V_2$ such that $\{u, v\} \in E_1 \iff \{\varphi(u), \varphi(v)\} \in E_2$.

Graph sum

Let $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ be two graphs such that $V_1 \cap V_2 = \emptyset$. We define $G_1 + G_2$, as the graph $G = (V, E)$ where:

- $V = V(G_1) \cup V(G_2)$
- $E = E(G_1) \cup E(G_2)$

Connectedness

A graph G is **connected** if it cannot be represented as the sum $G_1 + G_2$ for some non-empty graphs G_1 and G_2 . Otherwise, the graph is called **disconnected**. Equivalently, a graph is connected if there is a path between every pair of distinct vertices.

Connected Components

If a graph G can be expressed as a sum $G = G_1 + G_2 + \dots + G_n$ for some $n \in \mathbb{N}$, where each G_i is a connected graph, then each G_i is called a **connected component** of G .

Adjacency and Incidence

As previously mentioned, if $u, v \in V(G)$ and $uv \in E(G)$, we say that vertices u and v are **adjacent** (or **neighbours**).

If for a vertex $v \in V(G)$ and an edge $e \in E(G)$, we have $v \in e$ then we say that vertex v is **incident** to edge e .

Vertex Degree

For a vertex $v \in V(G)$, we define its **degree**, as:

$$\deg_G(v) = |\{u \in V(G) : vu \in E(G)\}|$$

Isolated and Leaf Vertices

- If $\deg_G(v) = 0$, then vertex v is called **isolated**.
- If $\deg_G(v) = 1$, then vertex v is called a **leaf**.

Degree Sequence

Let G be a graph with vertices $V(G) = \{v_1, v_2, \dots, v_n\}$. Then $\text{sort}[\deg(v_1), \dots, \deg(v_n)]$ is called the **degree sequence** of G

Handshaking Lemma

In any graph $G = (V, E)$, the sum of the degrees of all vertices is equal to twice the number of edges:

$$\sum_{v \in V(G)} \deg_G(v) = 2|E(G)|$$

Proof Sketch. Each edge $e = \{u, v\}$ contributes exactly 1 to the degree of vertex u and exactly 1 to the degree of vertex v . Therefore, when summing all degrees, each edge is counted twice. $\square$

Corollary 2. *In any graph G , the number of vertices with an odd degree is even.*

Subgraphs

A **subgraph** H of a graph G is a graph $H = (V_1, E_1)$, such that

- $V_1 \subseteq V(G)$
- $E_1 \subseteq E(G)$